SIMATS ENGINEERING

Department of Computer Science and Engineering

CSA10 – SOFTWARE ENGINEERING

ASSIGNMENT 2 – CO2

Analyze Requirements and Design a Scalable System Architecture for the Order Fulfilment Platform

CO2: Analyze software requirements to design scalable and efficient system architectures using collaborative tools like Jira and Confluence, along with structured modeling techniques such as UML diagrams and architectural design patterns.

Student Name	ATHIN SIVA.S	**Register Number**	192524237
Department / Section	AIDS	**Academic Year / Semester**	2025
Faculty Name		**Nature of Assignment**	Individual
Duration	1 Week	**Total Marks**	50
Date of Submission	5/11/2026	**Date of Evaluation**	

MARK SHEET (Rubric-based Evaluation)

Task	Task Title	Max	Marks Awarded	Remarks
1	Stakeholder Investigation and Conflict Resolution	5		
2	Prioritized and Testable Requirements Specification	8		
3	UML Modeling and Requirements Traceability	7		
4	Alternative Architectures and Justified Design	8		
5	Critical-Behaviour Validation and Design Revision	7		
6	Day-4 Architecture Redesign for Expansion and Partial Failure	10		
7	Individual Technical Reflection	5		
	TOTAL	50		

OVERALL REMARKS

Student Signature	**Faculty Signature**	**Program Director**

MANDATORY SUBMISSION EVIDENCE CHECK

Task	Evidence Expected	Status	Note
1	Stakeholder questions and findings, at least three conflicts and justified resolutions, actors, system boundary and exclusions.		
2	Prioritized functional and non-functional requirements with unique IDs, acceptance criteria, dependencies and traceability.		
3	UML use-case, class, sequence and state-machine models, plus component/deployment views and requirements traceability matrix.		
4	At least two architectural alternatives, decision rationale, component responsibilities, data ownership, interfaces and design patterns.		
5	Validation scenarios for order/refund invariants, duplicate callbacks, stock concurrency, authorization and failure behaviour; expected and observed results.		

6	Impact analysis, revised requirements and architecture diagrams, synchronization rules, partial-failure handling and comparative validation findings.		
7	Reflection on key requirement, stakeholder conflict, discovered inconsistency, remaining trade-off and further evidence needed.		

SUBMISSION CHECKLIST (not separately marked)

Requirement	Status
All questions (Q1–Q7) answered with required evidence	☐ Yes ☐ Partly ☐ No
Stakeholder investigation, conflicts and system boundary documented (Q1)	☐ Yes ☐ Partly ☐ No
Prioritized, testable requirements with acceptance criteria included (Q2)	☐ Yes ☐ Partly ☐ No
Required UML diagrams and traceability matrix included (Q3)	☐ Yes ☐ Partly ☐ No
Architectural alternatives compared and selected design justified (Q4)	☐ Yes ☐ Partly ☐ No
Critical behaviours and invariants validated with evidence (Q5)	☐ Yes ☐ Partly ☐ No
Day-4 redesign addresses expansion and partial failure (Q6)	☐ Yes ☐ Partly ☐ No
Individual reflection covers decisions, trade-offs and limitations (Q7)	☐ Yes ☐ Partly ☐ No
Jira / Confluence records and modelling artefacts included where applicable	☐ Yes ☐ Partly ☐ No
Assumptions, simulations and illustrative evidence clearly labelled	☐ Yes ☐ Partly ☐ No
No unfilled placeholders; requirements, diagrams and validation remain consistent	☐ Yes ☐ Partly ☐ No
Viva readiness: able to trace requirements and defend architecture decisions	☐ Yes ☐ No

Task 1 Investigate Stakeholder Needs and Resolve Conflicts

SIMULATED *All stakeholder responses below are simulated; no real interviews were held. They are labelled as such throughout.*

1.1 Investigation questions

Five questions target requirements that were unresolved at the start (the brief required at least three).

#	Question	Why it is unresolved
Q1	How does a picker learn that a reserved unit is no longer available?	Stock can change between pick-list creation and shelf visit; the brief does not say who is told.
Q2	Who may approve a refund above the standard limit, and what is the limit?	Refund control and refund speed conflict; the limit is not stated.
Q3	Which order details may a support agent view?	Support needs visibility, but payment data and contact details are restricted.
Q4	Who is responsible when the local order is stored but the courier request fails?	Courier API may be down; ownership of the retry is undefined.
Q5	What happens if a payment callback arrives after the reservation has expired?	Late payment and resale of the unit can collide.

1.2 Findings (simulated responses)

Stakeholder	Simulated response	Finding that shapes the specification
Picker (Q1)	"I only find out at the shelf. I want the pick list to be right before I walk."	Pick list contains only paid, reserved items (FR07); reservations expire visibly (FR08).
Finance reviewer (Q2)	"Anything over the limit comes to me. I need to see who asked and why."	Approval is a separate state from execution (FR10); every step is audited (FR12). Limit is an open question (U1).
Support agent (Q3)	"I need status and a way to contact the customer, not card details."	Masked contact, no payment data in support view (FR01, NFR04).
Store manager (Q4)	"No order may be lost because the courier is down, and reports must reconcile."	Durable integration intent (FR11); availability target under outage (NFR08).
Customer (Q5)	"If I paid I want my order or my money back, not silence."	Late-payment path: re-reserve or flag for refund (FR08).
Administrator	"I must change regional rules without a code release."	Versioned regional policies (FR13, Task 6).

1.3 Stakeholder analysis

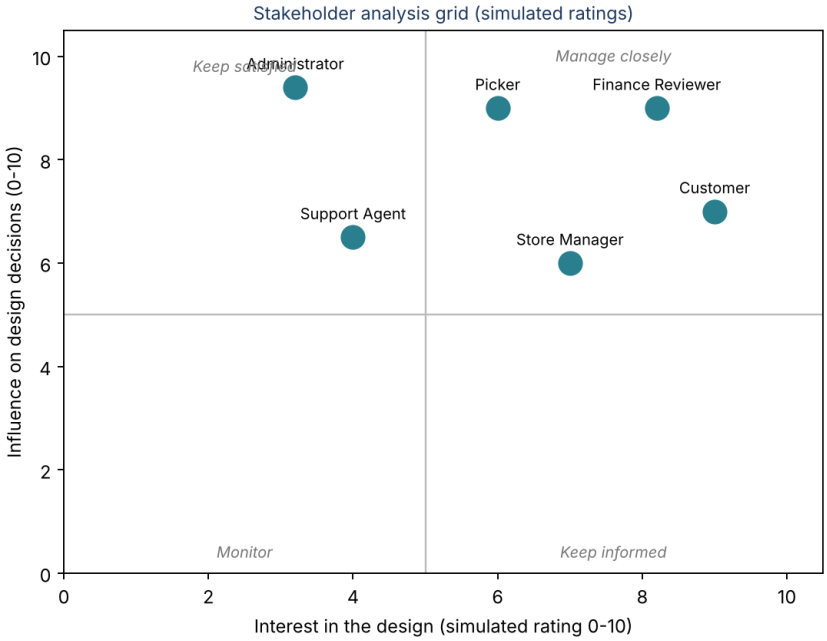


Figure 1.1 Influence/interest grid. Ratings are simulated judgements, not measurements.

Stakeholder	Primary need	Risk if ignored	Permissions (server-enforced)
Customer	Fast checkout, clear tracking	Abandoned orders	Place order; read own order
Picker	Accurate pick list, reliable reservation	Wasted picking trips	Confirm pick; dispatch
Support agent	Readable order and return status	Slow support; privacy breach	Masked read; request refund; start return
Finance reviewer	Auditable approvals	Fraud, unaudited refunds	Approve and execute refunds
Store manager	Consolidated sales, stock, refund reports	Wrong decisions	Read reports; approve small refunds
Administrator	Configurable regions and users	Release dependency	Write policies; manage users

1.4 Conflicts and resolutions

Conflict	Parties	Resolution and justification	Acceptance implication
C1 Checkout speed vs stock accuracy	Customer vs Picker	Reserve stock inside the order transaction with a 15-minute expiry. Checkout is slightly slower, but overselling the last unit costs more than a few extra milliseconds.	Two simultaneous buyers of the last unit: one order, one "out of stock".
C2 Support visibility vs customer privacy	Support vs Customer	Scoped, masked support view; card data never stored. Support gets what it needs to help, nothing more.	Cross-customer API read is rejected; support view has no payment data.
C3 Instant payment confirmation vs availability	Customer vs Engineer	Commit the order locally as Payment Pending; mark Paid only on a verified callback. A browser success page is not proof of payment.	Lost or repeated callback cannot lose the order or show Paid early.
C4 Fast refunds vs finance control	Support vs Finance	Agents request; finance (or a manager below the limit) approves; execution needs prior approval.	Agent or customer cannot approve; execution without approval is refused.

Conflict	Parties	Resolution and justification	Acceptance implication
C5 Automation vs human responsibility	Picker/Engineer vs Finance	Scanner events that would drive stock below reserved units are flagged for human correction, never auto-applied.	Conflicting scan lands in a discrepancy list; stock unchanged.

1.5 System boundary, actors and exclusions

Item	Content
Inside the boundary	Storefront interface, REST API, relational persistence, regional policy rules, integration worker (outbox), audit and reporting views.
Actors	Customer, Picker, Support Agent, Finance Reviewer, Store Manager, Administrator.
External systems	Payment gateway (sends signed callbacks), courier API, build server (GitHub + Jenkins). They do not share the organisation's permission model, so every inbound message is verified.
Inputs from outside	Warehouse scanners (unique scan events) and human reviewers (approval decisions).
Excluded from release 1	Autonomous refund decisions, ML demand forecasting (low-stock alerts are fixed-threshold rules, not ML), public API, live payment or courier access.

Stakeholder requests deliberately not turned into requirements: support asking to see full card numbers (conflicts with NFR04); a customer wish for a guaranteed unit while browsing (reservation happens only at checkout); a picker request to overwrite stock counts directly (stock corrections need human approval); a manager request for demand forecasting (out of scope).

1.6 Code: roles, permissions and the masked support view

The permission matrix and masked read below are taken unchanged from the test model (Appendix A). Identity comes from the server-side user record, never from a role chosen in the UI.

fulfiliq_test_model.py - permission matrix

```
ROLE_PERMS = {
    "customer": {"order.place", "order.read own"},
    "picker": {"pick.confirm", "order.dispatch", "picklist.read"},
    "support": {"order.read_masked", "refund.request", "return.start"},
    "finance": {"order.read_masked", "refund.approve", "refund.execute"},
    "manager": {"report.read", "refund.approve.small"},
    "admin": {"policy.write", "user.manage"},
}
```

fulfiliq_test_model.py - read_order (customer / support views)

```
    # ---------- FR01 / privacy ----------
    def read_order(self, user, order_id):
        o = self.orders[order_id]
        role = self.users.get(user, {}).get("role")
        if role == "customer":
            if o["owner"] != user:
                raise Rejected("not your order")
            return {k: o[k] for k in ("id", "state", "total")}
        if role in ("support", "finance"):
            name, domain = o["email"].split("@")
            return {"id": o["id"], "state": o["state"],
                    "contact": name[0] + "***@" + domain}   # masked, no payment data
        raise Rejected("role may not read orders")
```

MODEL-RUN *Behaviour demonstrated by scenario T6 (Task 5): cross-customer read rejected, support view masked, approval attempts by customer/support/picker rejected.*

Task 2 Prioritized and Testable Requirements Specification

2.1 Tool arrangement: options and selection

Criterion	A Jira + Confluence	B GitHub Issues + Markdown specs in repo	C Spreadsheet + shared document
Traceability	Issue keys link to Confluence pages and commits; JQL queries	Issue/PR links, but traceability matrix is manual	Manual cross-references only
Collaboration	Comments, mentions, review on both tools	Pull-request review	Comments; hard to control concurrent edits
Version history	Page history and issue history	Full Git history of every spec change	Document history only
Diagram support	Macros and image attachments; source not versioned	Diagram source (DOT/PlantUML) versioned with text	Images pasted in
Accessibility	Web-based, keyboard operable, steep learning curve	Plain text, screen-reader friendly	Familiar tools
Cost	Free tier for small teams; paid beyond limits	Free for the project	Free

Selected: Jira for requirement records and Confluence for the specification (option A), because project planning already lives in Jira and issue keys link directly to pages. **Limitations and mitigations:** diagram source is not versioned in Confluence, so the DOT/PlantUML sources are also committed to GitHub (option B's strength); the free tier limits users, which is acceptable for an individual assignment; links between a page and an issue can drift, so the traceability script in Task 3 re-checks the matrix against the backlog.

2.2 Prioritization method

MoSCoW adjusted by customer-harm risk. A requirement is **Must** if failure causes loss of money, stock integrity, privacy, or loss of an order. It is **Should** if failure degrades efficiency but the order lifecycle stays correct. Dependencies break ties: a requirement that other Musts rely on is never lower than them. Result: 18 Must, 5 Should, 0 Could (Could items such as forecasting are excluded in Task 1).

2.3 Functional requirements

ID	Requirement	Source	Priority	Acceptance criterion
FR01	Authenticate and authorize requests	Administrator; Finance Reviewer	Must	Unauthorized identity cannot modify protected records; role comes from session not UI
FR02	Place order	Customer	Must	Required fields and positive quantities validated; price computed on server
FR03	Reserve stock	Picker; Customer	Must	Reservation commits with the order; for the last unit exactly one concurrent buyer succeeds
FR04	Calculate price and tax	Store Manager	Must	Approved test cases match regional price and tax table; client-supplied price ignored
FR05	Confirm payment	Finance Reviewer	Must	Verified callback with one gateway+event id creates at most one payment; unverified rejected
FR06	Enforce order lifecycle	Support Agent; Picker	Must	Placed cannot jump to Dispatched; dispatch needs confirmed payment and pick confirmation
FR07	Generate pick list	Picker	Should	Picker sees only paid reserved items for assigned orders
FR08	Expire reservations	Customer; Picker	Should	Unpaid reservation released after 15 min exactly once; late callback handled explicitly

ID	Requirement	Source	Priority	Acceptance criterion
FR09	Dispatch and track	Customer	Must	Courier reference stored; customer sees permitted milestones only
FR10	Process return and refund	Finance Reviewer; Support Agent	Must	Refund needs eligible return; approval and execution are distinct states
FR11	Synchronize courier and gateway status	Engineer	Must	Durable integration intent survives retry and restart
FR12	Audit and report	Store Manager; Finance Reviewer	Should	Every authorized transition has actor time version; counts reconcile
FR13	Regional policy rules (Day-4)	Administrator	Must	Tax shipping and refund limit versioned per region; event records policy version
FR14	Offline scanner buffer (Day-4)	Picker	Must	Bounded queue; stable event id; server dedupes; no silent drop
FR15	Server-validated queued approval (Day-4)	Finance Reviewer	Must	Queued approval shown Pending until server validation

2.4 Non-functional requirements

ID	Quality	Source	Priority	Measurable target (operating condition and threshold)
NFR01	Checkout response time	Customer	Must	p95 < 800 ms for checkout at 50 concurrent synthetic users; record errors and hardware
NFR02	Consistency	Engineer	Must	Duplicate callback and concurrent reservation tests cause no second effect and no negative stock
NFR03	Recovery	Administrator	Must	Recovery within 15 min in test environment with stated RPO
NFR04	Security	Administrator	Must	All six roles tested for permitted and prohibited critical actions; no card numbers stored
NFR05	Usability	Customer; Support Agent	Should	Purchase and tracking operable by keyboard; status as text not colour alone
NFR06	Portability	Engineer	Should	Tagged source and documented config reproduce journey on clean host
NFR07	Expansion workload (Day-4)	Store Manager	Must	Design target 500 users and 1000 events/min across 12 regions; verify by load test
NFR08	Availability under integration outage	Store Manager; Customer	Must	During a 30-minute simulated courier or gateway outage 100% of valid orders are stored locally as Pending; API availability target 99.5% monthly (planning assumption)

All numeric targets are **proposed acceptance targets**, not measured results. Before measuring, specify dataset size, warm-up, network conditions and test duration.

Vague statement	Replacement
"Checkout should be fast"	NFR01: p95 below 800 ms for checkout at 50 concurrent synthetic users; record errors and hardware.
"The system should be secure"	NFR04: all six roles tested for permitted and prohibited critical actions; no card numbers stored.
"The system should be available"	NFR08: during a 30-minute simulated courier/gateway outage, 100% of valid orders are stored locally as Pending.
"It should be easy to use"	NFR05: purchase and tracking journey operable by keyboard; status shown as text, not colour alone.

2.5 Order rules

Rule	Statement	Enforced by	Status
Price and tax	Total = server-side price x quantity x (1 + regional tax). Client prices ignored.	FR04	Model T8

Rule	Statement	Enforced by	Status
Stock reservation	Reservation commits with the order; available = on_hand - reserved, never below zero; expiry 15 min.	FR03, FR08	Model T1, T7
Ownership	Every order has exactly one owner; only the owner (or a masked staff view) can read it.	FR01	Model T6
Permitted status changes	Placed, Paid, Packed, Dispatched, Delivered, Closed; Cancelled only before payment.	FR06	Model T4
Dispatch deadline	24 hours after payment confirmation, measured in UTC.	FR09	Planned (scheduler alert not in model)
Duplicate callbacks	A repeated gateway event returns the existing result.	FR05	Model T2, DB-demo
Refund approval	Refunds need approval; above the assumed limit a finance reviewer must approve.	FR10	Model T3, T6

2.6 Assumptions and unresolved questions

ID	Assumption / unresolved question
A1	Synthetic data only; no real customer or card data.
A2	Planning workload: 50 users and 100 events/min (release 1); 500 users and 1,000 events/min (Task 6). Design assumptions, not capacity.
A3	Peak load is assumed to be 5x average.
A4	Gateway and courier are simulated through controlled test interfaces; their status is disclosed.
U1	Unresolved: refund approval limit (assumed 100.00).
U2	Unresolved: reservation expiry length (assumed 15 minutes).
U3	Unresolved: regional return windows.
U4	Unresolved: retention period for customer addresses.

2.7 How two conflicts shaped the specification

- **C1 (speed vs stock accuracy)** produced FR03 (reservation inside the order transaction) and FR08 (15-minute expiry released exactly once), and accepted a slightly slower checkout in NFR01.
- **C4 (fast refunds vs finance control)** produced FR10 (approval and execution are distinct states), FR15 (queued approvals stay Pending), and the role checks in NFR04.
- **C3 (instant confirmation vs availability)** produced FR05 (verified callback only) and NFR08 (orders stored as Pending during outages).

2.8 Code and evidence

The backlog is held as requirements.csv; a script checks it and generates a Jira-importable file. The pricing rule (FR04) is implemented separately with exact decimal arithmetic.

requirements.csv (first rows)

```
ID,Type,Summary,Source,Priority,Acceptance criterion,Component,Scenario
FR01,Functional,Authenticate and authorize requests,Administrator; Finance
    Reviewer,Must,Unauthorized identity cannot modify protected records; role comes from session not
    UI,Authentication module,T6
FR02,Functional,Place order,Customer,Must,Required fields and positive quantities validated; price
    computed on server,Order and checkout,T1
FR03,Functional,Reserve stock,Picker; Customer,Must,Reservation commits with the order; for the
    last unit exactly one concurrent buyer succeeds,Stock module,T1
... (23 rows in total)
```

validate_requirements.py

```
"""Checks the backlog: unique ids, >=12 FR, >=6 NFR, every row has acceptance + scenario,
and writes a Jira-importable CSV."""
import csv, collections
rows = list(csv.DictReader(open("requirements.csv")))
ids = [r["ID"] for r in rows]
fr  = [r for r in rows if r["Type"] == "Functional"]
nfr = [r for r in rows if r["Type"] == "Non-functional"]
assert len(ids) == len(set(ids)), "duplicate IDs"
assert all(r["Acceptance criterion"] and r["Source"] and r["Priority"] for r in rows)
print(f"requirements total : {len(rows)}  (FR={len(fr)}, NFR={len(nfr)})")
print("priority split     :", dict(collections.Counter(r['Priority'] for r in rows)))
print("rows missing scenario:", [r["ID"] for r in rows if not r["Scenario"]] or "none")
assert len(fr) >= 12 and len(nfr) >= 6
with open("jira_import.csv", "w", newline="") as f:
    w = csv.writer(f)
    w.writerow(["Summary", "Issue Type", "Priority", "Labels", "Description"])
    for r in rows:
        pr = {"Must": "Highest", "Should": "High"}[r["Priority"]]
        w.writerow([f'{r["ID"]} {r["Summary"]}', "Story" if r["Type"] == "Functional" else "Task",
                    pr, r["Component"].replace(" ", "_"), "AC: " + r["Acceptance criterion"]])
print("jira_import.csv written ->", len(rows), "issues")
print("VALIDATION PASS")
```

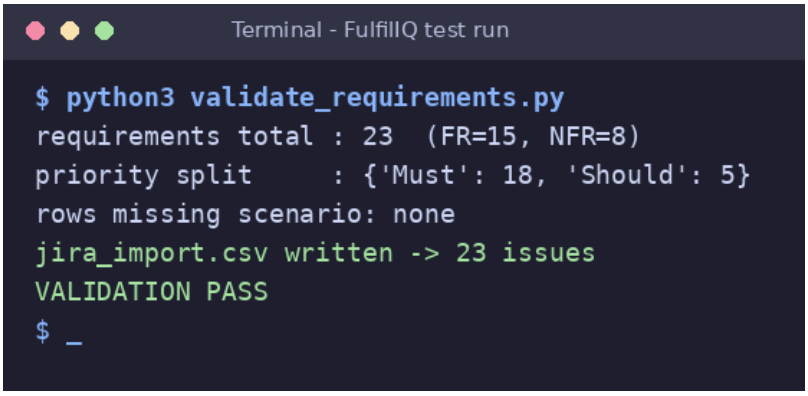


Figure 2.1 Output of validate_requirements.py (real run).

MODEL-RUN *Real terminal output from the script above.*

pricing.py - FR04 price and tax

```
"""FR04: server-side price and tax. Client-supplied prices are never used."""
from decimal import Decimal, ROUND_HALF_UP
TAX = {"R1": Decimal("0.18"), "R2": Decimal("0.12"), "R3": Decimal("0.05")}
CATALOG = {"SKU-1": Decimal("50.00"), "SKU-2": Decimal("19.99")}

def order_total(sku, qty, region, client_price=None):
    if qty <= 0:
        raise ValueError("quantity must be positive")
    net = CATALOG[sku] * qty                      # client_price deliberately ignored
    return (net * (1 + TAX[region])).quantize(Decimal("0.01"), ROUND_HALF_UP)

if __name__ == "__main__":
    cases = [("SKU-1", 1, "R1", Decimal("59.00")), ("SKU-1", 2, "R2", Decimal("112.00")),
             ("SKU-2", 3, "R3", Decimal("62.97")), ("SKU-1", 1, "R1", Decimal("59.00"))]
    for sku, q, reg, expected in cases:
        got = order_total(sku, q, reg, client_price=0.01)   # tampered client price
        print(f"{sku} x{q} {reg}: expected {expected} got {got} ->",
              "PASS" if got == expected else "FAIL")
```

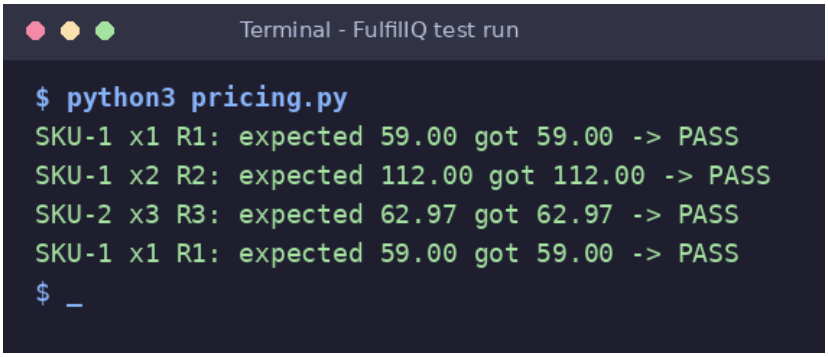


Figure 2.2 Output of pricing.py, including a tampered client price (real run).

Jira | FulfilIQ (FOI) | Backlog

Backlog - imported from jira_import.csv (23 issues)

Key	Summary	Type	Priority	Component label	Status
FOI-1	FR01 Authenticate and authorize requests	Story	^^ Highest	Authentication_module	DONE (model)
FOI-2	FR02 Place order	Story	^^ Highest	Order_and_checkout	DONE (model)
FOI-3	FR03 Reserve stock	Story	^^ Highest	Stock_module	DONE (model)
FOI-4	FR04 Calculate price and tax	Story	^^ Highest	Order_and_checkout	DONE (model)
FOI-5	FR05 Confirm payment	Story	^^ Highest	Payment_callback_handler	DONE (model)
FOI-6	FR06 Enforce order lifecycle	Story	^^ Highest	Order_command_service	DONE (model)
FOI-7	FR07 Generate pick list	Story	^ High	Fulfilment_module	DONE (model)
FOI-8	FR08 Expire reservations	Story	^ High	Stock_module_+_scheduler	DONE (model)
FOI-9	FR09 Dispatch and track	Story	^^ Highest	Fulfilment_module	DONE (model)
FOI-10	FR10 Process return and refund	Story	^^ Highest	Refund_approval_module	DONE (model)
FOI-11	FR11 Synchronize courier and gateway status	Story	^^ Highest	Outbox_worker	DONE (model)
FOI-12	FR12 Audit and report	Story	^ High	Reporting_views	DONE (model)
FOI-13	FR13 Regional policy rules (Day-4)	Story	^^ Highest	Policy_module	DONE (model)
FOI-14	FR14 Offline scanner buffer (Day-4)	Story	^^ Highest	Scan_sync_endpoint	DONE (model)
FOI-15	FR15 Server-validated queued approval (Day-4)	Story	^^ Highest	Refund_approval_module	DONE (model)
FOI-16	NFR01 Checkout response time	Task	^^ Highest	API_+_database	TO DO
FOI-17	NFR02 Consistency	Task	^^ Highest	Database_constraints	DONE (model)

ILLUSTRATIVE MOCK-UP generated from requirements.csv - replace with a screenshot of your own Jira board before submission.

Figure 2.3 Requirement records in the issue tracker (illustrative mock-up generated from the backlog).

Confluence | FulfilIQ space | Requirements Specification

FulfilIQ Home
- 1 Stakeholder analysis
- **2 Requirements spec (v1.3)**
- 3 UML models
- 4 Architecture Decision Records
  - ADR01 Relational storage
  - ADR02 Durable outbox
  - ADR03 Server-side authz
- 5 Validation scenarios
- 6 Day-4 redesign

2 Requirements specification

Version 1.3 | Owner: student | History: 7 versions | Linked Jira project: FOI

Assumptions: refund limit 100.00 (U1), reservation expiry 15 min (U2), peak = 5 x average. Targets are NOT measured capacity.
Unresolved: regional return windows (U3); retention period for customer addresses (U4).

ID	Requirement	Pri.	Acceptance criterion
FR01	Authenticate and authorize requests	Must	Unauthorized identity cannot modify protected records; role ...
FR02	Place order	Must	Required fields and positive quantities validated; price com...
FR03	Reserve stock	Must	Reservation commits with the order; for the last unit exactl...
FR04	Calculate price and tax	Must	Approved test cases match regional price and tax table; clie...
FR05	Confirm payment	Must	Verified callback with one gateway+event id creates at most ...
FR06	Enforce order lifecycle	Must	Placed cannot jump to Dispatched; dispatch needs confirmed p...
FR07	Generate pick list	Should	Picker sees only paid reserved items for assigned orders
FR08	Expire reservations	Should	Unpaid reservation released after 15 min exactly once; late ...
FR09	Dispatch and track	Must	Courier reference stored; customer sees permitted milestones...
FR10	Process return and refund	Must	Refund needs eligible return; approval and execution are dis...
FR11	Synchronize courier and gateway status	Must	Durable integration intent survives retry and restart
FR12	Audit and report	Should	Every authorized transition has actor time version; counts r...
FR13	Regional policy rules (Day-4)	Must	Tax shipping and refund limit versioned per region; event re...
FR14	Offline scanner buffer (Day-4)	Must	Bounded queue; stable event id; server dedupes; no silent dr...
FR15	Server-validated queued approval (Day-4)	Must	Queued approval shown Pending until server validation

ILLUSTRATIVE MOCK-UP built from requirements.csv - replace with a screenshot of your own Confluence page before submission.

Figure 2.4 Specification page in the documentation tool (illustrative mock-up generated from the backlog).

SIMULATED *Figures 2.3 and 2.4 are generated mock-ups of the tracker and wiki. Replace them with screenshots of your own Jira and Confluence before submission.*

Task 3 UML Models and Traceability

The models below are consistent with each other and with the requirements. Diagrams were produced from source code (matplotlib and Graphviz DOT) so they can be regenerated and versioned.

3.1 Use case diagram

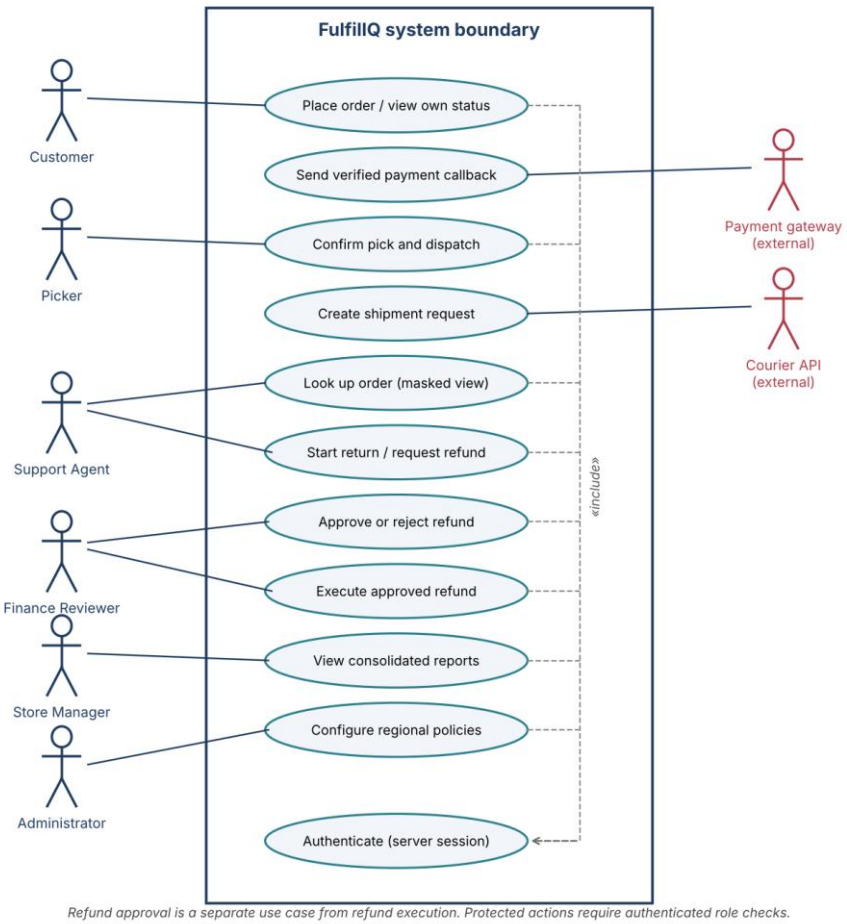


Figure 3.1 Use case model. Refund approval and execution are separate use cases; authentication is included by every protected use case.

3.2 Class diagram

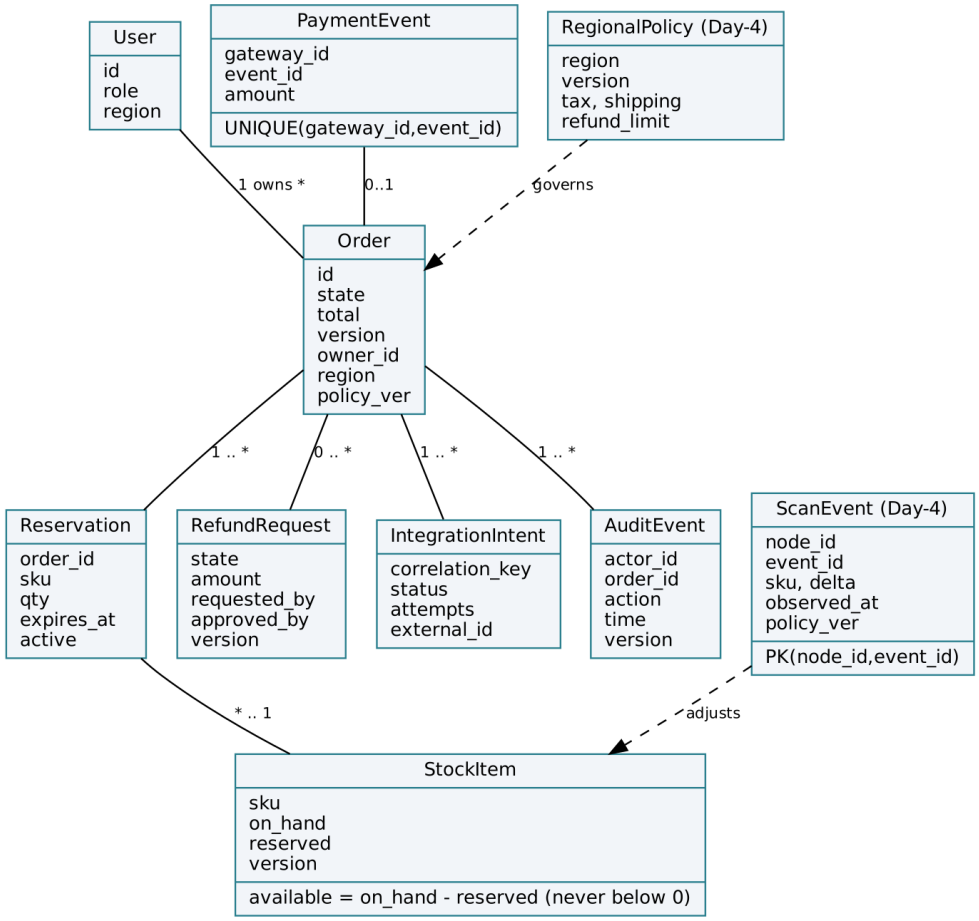


Figure 3.2 Domain class model. Constraints: unique (gateway_id, event_id); unique correlation_key; on_hand - reserved never below zero.

3.3 Sequence diagram: payment callback to courier synchronization

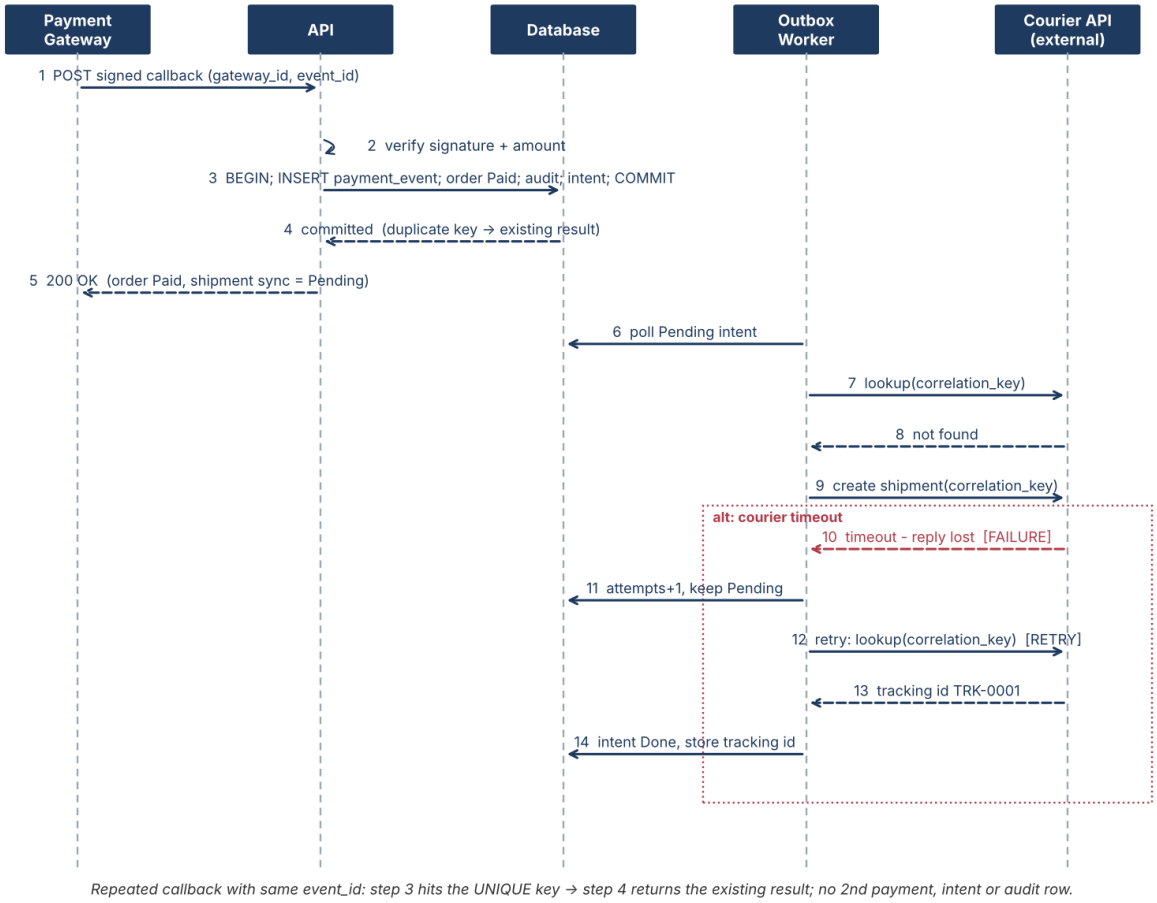


Figure 3.3 Payment callback, order confirmation, shipment intent and courier synchronization, including a timeout and retry.

The API answers the gateway as soon as the payment, order state, audit row and shipment intent are committed together. The worker contacts the courier afterwards, so a courier outage never rolls back a paid order. On timeout the worker looks the shipment up by correlation key before retrying and therefore never creates a second shipment.

3.4 State machines

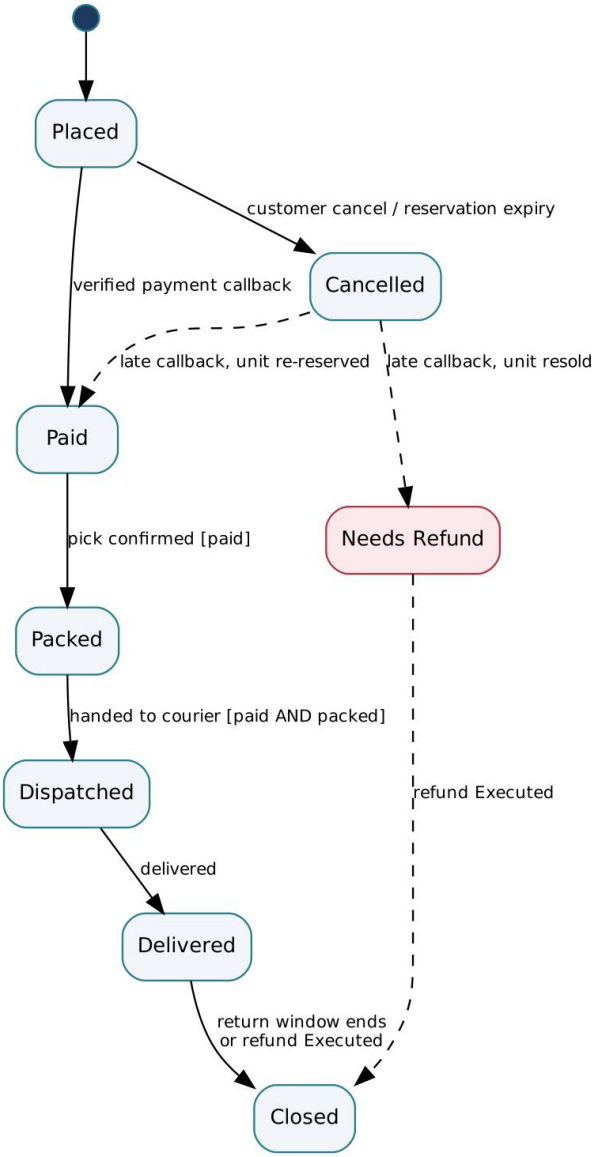


Figure 3.4 Order state model with guarded transitions (late-payment paths added after validation, Task 5).

Each command checks role, expected version, required evidence and current state. Courier states, order states and warehouse-board states are mapped, not assumed identical.

Refund approval, rejection, execution and rollback

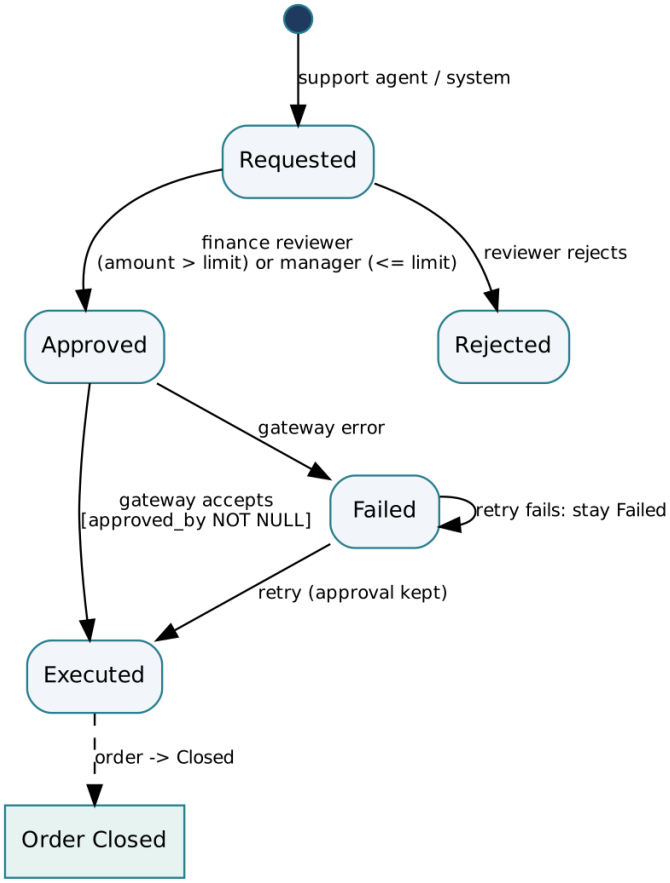


Figure 3.5 Refund state model.

- **Approval:** only a finance reviewer (any amount) or a manager (amount at or below the limit) can move Requested to Approved.
- **Rejection:** a reviewer can reject; the order is unchanged and no money moves.
- **Execution:** allowed only when approved_by is set. Success moves the order to Closed.
- **Rollback:** a gateway failure sets the refund to Failed. The order stays Delivered (or Needs Refund), not Closed, and the **prior approval is kept**, so a controlled retry needs no new approval.

3.5 Traceability matrix

Req.	UML element	Component	Scenario	Status
FR01	Use case Authenticate; User.role	Authentication module	T6	model-run
FR02	Use case Place order; Order	Order and checkout	T1	model-run
FR03	Reservation, StockItem	Stock module	T1	model-run
FR04	Order.total; RegionalPolicy	Order and checkout	T8	model-run
FR05	PaymentEvent unique key; Seq 1-5	Payment callback handler	T2	model-run
FR06	Order state machine	Order command service	T4	model-run
FR07	Use case Confirm pick	Fulfilment module	T4	model-run
FR08	Reservation.expires_at; Placed to Cancelled	Stock module + scheduler	T7	model-run
FR09	Order tracking; Dispatched	Fulfilment module	T5	model-run
FR10	RefundRequest state machine	Refund approval module	T3	model-run
FR11	IntegrationIntent; Seq 6-14	Outbox worker	T5	model-run

Req.	UML element	Component	Scenario	Status
FR12	AuditEvent	Reporting views	T9	planned
FR13	RegionalPolicy (version)	Policy module	S2	model-run
FR14	ScanEvent (node, event_id)	Scan sync endpoint	S1	model-run
FR15	RefundRequest queued approval	Refund approval module	S3	model-run
NFR01	Quality attribute (cross-cutting)	API + database	L1	planned
NFR02	Quality attribute (cross-cutting)	Database constraints	T1;T2	model-run
NFR03	Quality attribute (cross-cutting)	Backup + deployment	L3	planned
NFR04	Quality attribute (cross-cutting)	Authentication module	T6	model-run
NFR05	Quality attribute (cross-cutting)	React UI	U1	planned
NFR06	Quality attribute (cross-cutting)	Docker images	D1	planned
NFR07	Quality attribute (cross-cutting)	API + queue + read models	S1;L1	planned
NFR08	Quality attribute (cross-cutting)	Outbox worker + database	T5	model-run

17 of 23 requirements are covered by an executed model scenario; 6 (NFR01, NFR03, NFR05, NFR06, NFR07, FR12) still need a deployed environment, load tool or UI test.

3.6 Inconsistencies found and corrected

#	Found by	Inconsistency	Correction
1	Sequence vs requirements	A browser "payment success" page was treated as confirming payment.	Payment is confirmed only by a verified signed callback (FR05); sequence step 1 is the only trigger.
2	Use case vs class model	A UI role selector appeared to change the user's permissions.	Identity and role come from the authenticated server session (FR01); User.role is server-owned.
3	State model vs refund scenario (T3)	A Failed refund could not be retried, and failure might erase approval.	Added Failed to Executed with approval kept; order not Closed until execution succeeds.
4	State model vs expiry walkthrough (T7)	A late callback could mark a Cancelled order Paid although the unit was resold.	Added Cancelled to Paid (re-reserve) and Needs Refund paths; FR08 and Figure 3.4 updated.

3.7 Code: diagram source and traceability checker

state.dot (Graphviz source of Figure 3.4, first lines)

```
digraph G{rankdir=TB;graph[fontname="DejaVu Sans",pad=0.2];node[fontname="DejaVu
    Sans",fontsize=11];edge[fontname="DejaVu Sans",fontsize=9];node[shape=box,style="rounded,filled",f
    illcolor="#F2F5F9",color="#2A7F8E"];start[shape=circle,width=.2,label="",style=filled,fillcolor="#
    1F3A5F"];
start->Placed;
Placed->Paid[label="verified payment callback"];
Paid->Packed[label="pick confirmed [paid]"];
Packed->Dispatched[label="handed to courier [paid AND packed]"];
Dispatched->Delivered[label="delivered"];
Delivered->Closed[label="return window ends\nor refund Executed"];
Placed->Cancelled[label="customer cancel / reservation expiry"];
Cancelled->Paid[label="late callback, unit re-reserved",style=dashed];
Cancelled->NeedsRefund[label="late callback, unit resold",style=dashed];
NeedsRefund[label="Needs Refund",fillcolor="#FBE9EB",color="#B23A48"];
NeedsRefund->Closed[label="refund Executed",style=dashed];
}
```

traceability.py

```
"""Builds Requirement -> UML element -> Component -> Scenario matrix and reports status."""
import csv, re
UML = {
 "FR01": "Use case: Authenticate; User.role", "FR02": "Use case: Place order; Order",
 "FR03": "Reservation, StockItem", "FR04": "Order.total; RegionalPolicy",
 "FR05": "PaymentEvent (unique key); Seq msg 1-3", "FR06": "Order state machine",
 "FR07": "Use case: Confirm pick", "FR08": "Reservation.expires_at; State Placed->Cancelled",
 "FR09": "Order.tracking; state Dispatched", "FR10": "RefundRequest state machine",
```

```
 "FR11": "IntegrationIntent; Seq msg 6-9", "FR12": "AuditEvent",
 "FR13": "RegionalPolicy(version)", "FR14": "ScanEvent(node,event_id)",
 "FR15": "RefundRequest.pending_validation",
}
executed = set(re.findall(r"test_((?:T|S)\d+)", open("test_fulfiliq.py").read()))
rows = list(csv.DictReader(open("requirements.csv")))
print(f"{'REQ':6}{'UML element':44}{'Component':28}{'Scenario':10}Status")
gaps = 0
for r in rows:
    scs = r["Scenario"].split(";")
    ok = all(s in executed for s in scs)
    status = "model-run" if ok else "planned"
    gaps += (not ok)
    print(f"{r['ID']:6}{UML.get(r['ID'], '(quality
    attr.)'):44.43}{r['Component']:28.27}{r['Scenario']:10}{status}")
print(f"\n{len(rows)-gaps} requirements covered by an executed model scenario; "
      f"{gaps} still planned (need deployed environment or UI tests).")
```

Terminal - FulfillQ test run

```
$ python3 traceability.py
REQ   UML element                                Component                 Scenario  Status
FR01  Use case: Authenticate; User.role          Authentication module     T6        model-run
FR02  Use case: Place order; Order               Order and checkout        T1        model-run
FR03  Reservation, StockItem                     Stock module              T1        model-run
FR04  Order.total; RegionalPolicy                Order and checkout        T8        model-run
FR05  PaymentEvent (unique key); Seq msg 1-3     Payment callback handler  T2        model-run
FR06  Order state machine                        Order command service     T4        model-run
FR07  Use case: Confirm pick                     Fulfilment module         T4        model-run
FR08  Reservation.expires_at; State Placed->Cance Stock module + scheduler T7        model-run
FR09  Order.tracking; state Dispatched           Fulfilment module         T5        model-run
FR10  RefundRequest state machine                Refund approval module    T3        model-run
FR11  IntegrationIntent; Seq msg 6-9             Outbox worker             T5        model-run
FR12  AuditEvent                                 Reporting views           T9        planned
FR13  RegionalPolicy(version)                    Policy module             S2        model-run
FR14  ScanEvent(node,event_id)                   Scan sync endpoint        S1        model-run
FR15  RefundRequest.pending_validation           Refund approval module    S3        model-run
NFR01 (quality attr.)                            API + database            L1        planned
NFR02 (quality attr.)                            Database constraints      T1;T2     model-run
NFR03 (quality attr.)                            Backup + deployment       L3        planned
NFR04 (quality attr.)                            Authentication module     T6        model-run
NFR05 (quality attr.)                            React UI                  U1        planned
NFR06 (quality attr.)                            Docker images             D1        planned
NFR07 (quality attr.)                            API + queue + read models S1;L1     planned
NFR08 (quality attr.)                            Outbox worker + database  T5        model-run

17 requirements covered by an executed model scenario; 6 still planned (need deployed environment or UI tests).
$ _
```

Figure 3.6 Output of traceability.py (real run).

MODEL-RUN *Status column is computed by checking which scenario IDs exist as executed tests.*

Task 4 Alternative Architectures and Justified Design

4.1 Two alternatives

Alternative A: modular application with a worker. One deployable FastAPI application divided into modules (authentication, order and checkout, stock and reservation, payment callbacks, refunds, fulfilment and tracking, reporting) with one PostgreSQL database and one integration worker.
Alternative B: independent services with an event broker. Separate order, stock, payment, refund, tracking and reporting services, each with its own database, communicating through a message broker.

Dimension	A Modular application + worker	B Services + event broker
Components and responsibilities	Modules execute domain rules inside one process; worker sends gateway/courier requests from the outbox.	One service per business capability plus broker; each service owns its rules.
Communication	In-process calls between modules; REST from clients; database outbox to external systems.	Asynchronous events on the broker; REST at the edge.
Data ownership	One relational database; each module owns its tables by convention and code boundary.	Each service owns its database; others read through events or APIs.
Prevent duplicate payment	UNIQUE (gateway_id, event_id) in the same transaction that marks the order Paid.	Payment service keeps a processed-event table; downstream services must also deduplicate redelivered events.
Prevent overselling the last unit	Conditional stock update inside the order transaction: available stock must cover the quantity.	Stock service reserves; order service waits for a reservation event and compensates on failure.
Scalability	Replicate stateless API and worker; database is the shared limit.	Scale stock, payment and tracking independently.
Consistency	Order, stock, audit and intent commit atomically.	Eventual; needs idempotent handlers, ordering and compensation.
Availability	Single database is a central dependency; external outages absorbed by the outbox.	Partial failures are isolated, but broker and inter-service calls add failure modes.
Security	One trust boundary to audit; server-side role checks in one place.	Many internal trust boundaries; service-to-service authentication required.
Operational complexity	Few deployment units; one team can run it.	More moving parts, monitoring, tracing and failure diagnosis.
Likely failure points	Database outage; worker stalls; migrations affecting every module.	Broker outage; lost or reordered events; stuck compensations; version skew between services.
Maintenance	Module boundaries can erode without discipline; one release train.	Independent releases but contract management and duplicated infrastructure.

4.2 Comparison and selection

Criterion	Weight %	A score (1-5)	B score (1-5)
Consistency of order, stock, refund	25	5	3
Operational simplicity for a small team	25	5	2
Scalability headroom	15	3	5
Availability under partial failure	15	3	4
Security surface	10	4	4
Infrastructure cost	10	5	2
Weighted total	100	**4.30**	**3.20**

Weights and scores are the author's judgement for a small team and a modest deployment; they structure the argument and are not measurements. **Selected: Alternative A.** It keeps the most critical rules (no duplicate payment, no oversell, approval-gated refunds) inside single database transactions, and it matches a small team's capacity. Alternative B is reconsidered only when measured,

independent workload justifies it. A modular application is not automatically a set of microservices; the module boundaries keep that option open.

4.3 Component diagram

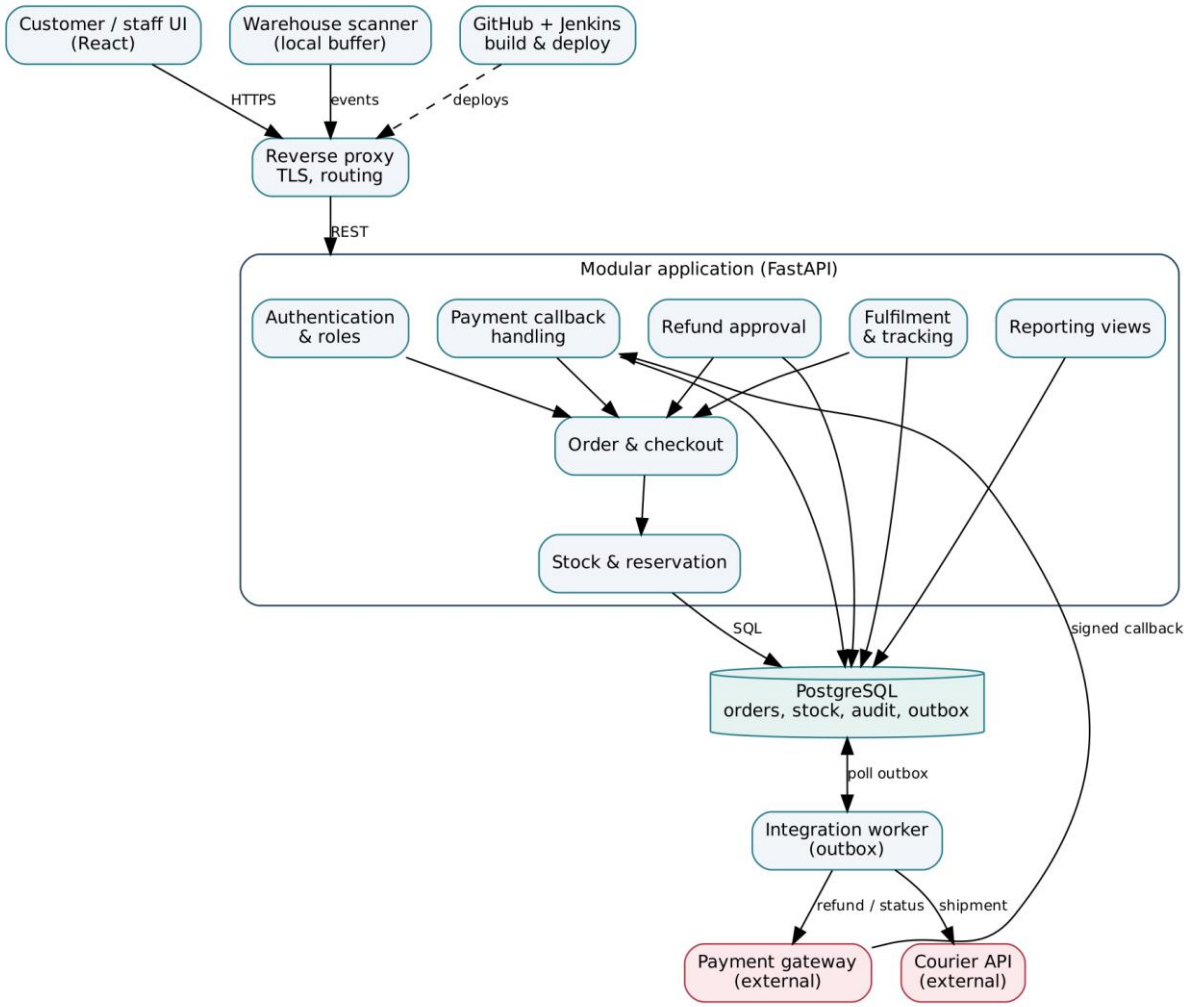


Figure 4.1 Selected architecture: component view.

The reverse proxy terminates TLS and routes to the stateless API. Scanners, the gateway and the courier cross trust boundaries and are validated at the API; external systems never write to the database directly.

4.4 Deployment diagram

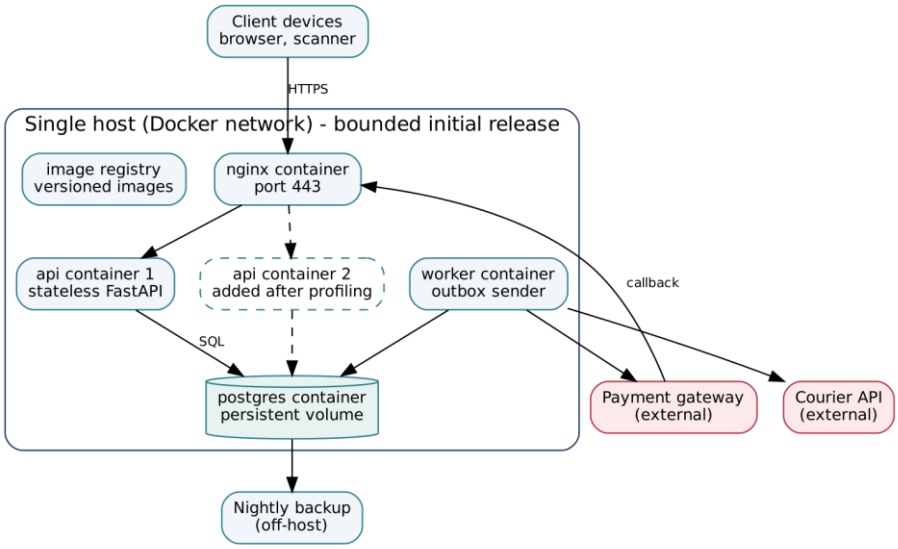


Figure 4.2 Deployment view for the bounded initial release (one host, Docker network).

4.5 Architectural decision records

ADR01 Relational storage for orders, stock, approvals, audit and intents

Field	Content
Problem	Order state, stock, refund approvals, audit and integration intents must change together without partial updates.
Alternatives	(1) PostgreSQL relational schema. (2) Document store with per-document writes. (3) Separate store per module.
Decision	PostgreSQL with constraints, transactions and foreign keys.
Justification	Constraints (CHECK, UNIQUE, foreign keys) enforce invariants in the database itself. A document store is not inherently incapable of transactions, so the choice rests on schema and operational fit, not capability.
Consequences	Schema migration work; one database is a central dependency (see Task 6); reporting must avoid competing with checkout.

ADR02 Durable outbox for gateway and courier integration

Field	Content
Problem	Local order recording must not depend on gateway or courier availability, and retries must not duplicate shipments.
Alternatives	(1) Call courier synchronously inside the request. (2) Fire-and-forget background call. (3) Transactional outbox plus worker.
Decision	Write an integration intent (correlation key, status, attempts) in the same transaction as the order change; a worker sends it and reconciles by correlation key.
Justification	Intent survives restart and outage; reconciling before retry prevents a second shipment (scenario T5).
Consequences	Delayed synchronization (shown as Pending); worker needs monitoring and a dead-letter review.

ADR03 Server-side authorization, idempotency keys and optimistic versioning

Field	Content
Problem	Hidden UI controls and blind updates cannot stop forged requests, stale overwrites or privilege escalation.
Alternatives	(1) Hide controls in the UI. (2) Role sent by the client. (3) Server-side role checks from the session, idempotency keys, version column.

Field	Content
Decision	Option 3 for every command: role, expected version, evidence and current state are checked on the server.
Justification	Closes the role-selector inconsistency (Task 3, #2) and the duplicate-callback and stale-update risks.
Consequences	Clients must handle conflict responses; every endpoint needs a permission mapping and tests.

ADR04 Modular application rather than microservices for release 1

Field	Content
Problem	Choose an architecture a small team can operate while keeping consistency.
Alternatives	Alternative A (modular) or Alternative B (services and broker).
Decision	Alternative A with enforced module boundaries.
Justification	Weighted comparison in 4.2; transactional consistency; fewer deployment units.
Consequences	Cannot scale modules independently; revisit when measured workload justifies a split.

4.6 Code: schema, callback endpoint and container layout

DESIGN *Reference code for the selected stack. It was not executed here (FastAPI and PostgreSQL are not installed in this environment); the same two database mechanisms are demonstrated with SQLite in Task 5.*

schema.sql (PostgreSQL)

```
-- PostgreSQL reference schema (core of ADR01 / ADR02 / ADR03). Not executed in this report.
CREATE TABLE stock_item (
  sku        TEXT PRIMARY KEY,
  on_hand    INT NOT NULL CHECK (on_hand >= 0),
  reserved   INT NOT NULL DEFAULT 0,
  version    INT NOT NULL DEFAULT 1,
  CHECK (reserved >= 0 AND reserved <= on_hand)          -- invariant 3
);
CREATE TABLE orders (
  id         BIGSERIAL PRIMARY KEY,
  owner_id   BIGINT NOT NULL,
  region     TEXT   NOT NULL,
  state      TEXT   NOT NULL CHECK (state IN ('Placed','Paid','Packed','Dispatched',
               'Delivered','Closed','Cancelled','Needs Refund')),
  total      NUMERIC(12,2) NOT NULL,
  policy_ver INT    NOT NULL,
  version    INT    NOT NULL DEFAULT 1                   -- optimistic locking
);
CREATE TABLE reservation (
  order_id   BIGINT REFERENCES orders(id),
  sku        TEXT   REFERENCES stock item(sku),
  qty        INT    NOT NULL CHECK (qty > 0),
  expires_at TIMESTAMPTZ NOT NULL,
  active     BOOLEAN NOT NULL DEFAULT TRUE,
  PRIMARY KEY (order_id, sku)
);
CREATE TABLE payment_event (
  gateway_id TEXT NOT NULL,
  event_id   TEXT NOT NULL,
  order_id   BIGINT REFERENCES orders(id),
  amount     NUMERIC(12,2) NOT NULL,
  PRIMARY KEY (gateway_id, event_id)                     -- invariant 1
);
CREATE TABLE refund_request (
  id BIGSERIAL PRIMARY KEY, order_id BIGINT REFERENCES orders(id),
  amount NUMERIC(12,2) NOT NULL,
  state TEXT NOT NULL CHECK (state IN ('Requested','Approved','Rejected','Executed','Failed')),
  requested_by BIGINT NOT NULL, approved_by BIGINT, version INT NOT NULL DEFAULT 1,
  CHECK (state NOT IN ('Executed') OR approved_by IS NOT NULL)   -- invariant 2
);
CREATE TABLE integration_intent (                        -- transactional outbox
  id BIGSERIAL PRIMARY KEY, order id BIGINT REFERENCES orders(id),
  correlation_key TEXT NOT NULL UNIQUE,
  status TEXT NOT NULL DEFAULT 'Pending', attempts INT NOT NULL DEFAULT 0, external_id TEXT
);
CREATE TABLE audit_event (
  id BIGSERIAL PRIMARY KEY, actor_id TEXT, order_id BIGINT, action TEXT NOT NULL,
  at TIMESTAMPTZ NOT NULL DEFAULT now(), version INT
);
CREATE TABLE scan_event (                                -- Day-4 deduplication
  node_id TEXT NOT NULL, event_id TEXT NOT NULL, sku TEXT NOT NULL, delta INT NOT NULL,
  observed_at TIMESTAMPTZ NOT NULL, policy_ver INT NOT NULL, stale BOOLEAN DEFAULT FALSE,
```

```
  PRIMARY KEY (node_id, event_id)
);
CREATE INDEX idx_orders_owner ON orders(owner_id, state);
CREATE INDEX idx_intent_pending ON integration_intent(status) WHERE status = 'Pending';
```

api_skeleton.py (FastAPI)

```
"""FastAPI reference skeleton for the payment callback (FR05). Reference code: not executed
    here."""
from fastapi import FastAPI, Depends, HTTPException, Header
import hmac, hashlib, psycopg

app = FastAPI()

def current_user(authorization: str = Header(...)):
    user = verify_session(authorization)                # identity from server session
    if user is None:
        raise HTTPException(401)
    return user                                         # never trust a UI role selector

def require(perm):
    def dep(user=Depends(current user)):
        if perm not in user.permissions:
            raise HTTPException(403, "forbidden")
        return user
    return dep

@app.post("/callbacks/payment")
def payment_callback(body: dict, x_signature: str = Header(...)):
    expected = hmac.new(SECRET, canonical(body), hashlib.sha256).hexdigest()
    if not hmac.compare digest(expected, x signature):
        raise HTTPException(400, "bad signature")
    with db.transaction() as tx:
        inserted = tx.execute(
            "INSERT INTO payment_event(gateway_id,event_id,order_id,amount) "
            "VALUES (%s,%s,%s,%s) ON CONFLICT DO NOTHING RETURNING 1",
            (body["gateway_id"], body["event_id"], body["order_id"], body["amount"])).fetchone()
        if inserted is None:                            # duplicate: return existing result
            return {"status": "duplicate_ignored"}
        tx.execute("UPDATE orders SET state='Paid', version=version+1 "
                   "WHERE id=%s AND state='Placed'", (body["order_id"],))
        tx.execute("INSERT INTO integration_intent(order_id, correlation_key) VALUES (%s,%s)",
                   (body["order_id"], f"ship-{body['order_id']}"))
        tx.execute("INSERT INTO audit event(actor id,order id,action) VALUES
    ('gateway',%s,'payment.confirmed')",
                   (body["order_id"],))
    return {"status": "paid", "shipment_sync": "Pending"}

@app.post("/refunds/{rid}/approve")
def approve(rid: int, user=Depends(require("refund.approve"))):
    ...  # UPDATE refund_request SET state='Approved', approved_by=%s WHERE id=%s AND
    state='Requested'
```

docker-compose.yml

```
services:
  nginx:   { image: nginx:1.27, ports: ["443:443"], depends_on: [api1] }
  api1:    { build: ./app, environment: { DATABASE_URL: postgresql://app@db/fulfiliq },
    depends on: [db] }
  api2:    { build: ./app, profiles: ["scaled"], environment: { DATABASE_URL:
    postgresql://app@db/fulfiliq } }  # added after profiling
  worker:  { build: ./app, command: python -m worker.outbox, depends_on: [db] }
  db:      { image: postgres:16, volumes: ["pgdata:/var/lib/postgresql/data"] }
volumes: { pgdata: {} }
```

Task 5 Validate Critical Behaviour and Revise the Design

5.1 Invariants

ID	Invariant	Enforced by
INV1	One gateway/event identity has at most one payment.	UNIQUE (gateway_id, event_id); FR05
INV2	A refund cannot execute without prior approval.	approved_by check; CHECK constraint; FR10
INV3	Available stock never goes below zero.	Conditional update; CHECK constraint; FR03
INV4	One accepted order command creates exactly one audit action in the same transaction.	Transaction boundary; FR12
INV5	A stale version cannot silently overwrite accepted state.	Optimistic version column; ADR03

These invariants do not give immutable evidence against a database administrator; access and storage controls still matter.

5.2 Operation contracts

Operation	Preconditions	Postconditions
Place order	Authenticated customer; positive quantity; matching stock version; available stock at least the quantity.	Order Placed; reserved count increased once; audit appended; a failed check leaves all state unchanged.
Confirm payment	Valid signature; known order; amount matches order total; unseen event identity.	Order Paid once; one shipment intent created; repeated identity returns the existing result with no new effect.
Execute refund	Role may execute; refund state Approved or Failed; approved_by set.	Success: refund Executed and order Closed. Failure: refund Failed, approval kept, order not Closed.

5.3 Validation scenarios: predictions first, then observations

Predictions were written as the expected results and assertions before each scenario was run. The terminal output in 5.4 shows what actually happened. "Observed" below is therefore a **demonstrated** result only for the labelled method; everything marked planned remains a prediction.

ID	Scenario	Prediction (written first)	Enforced by	Observed	Evidence
T1	Two customers buy the last unit	One order; second buyer rejected (stale version, then out of stock); stock not negative.	FR03, INV3	Model: 1 order, second rejected. SQLite, 20 threads: 1 order, 19 rejected.	MODEL-RUN DB-DEMO
T2	Repeated callback, including while the first is in flight	One payment, one intent, one audit entry.	FR05, INV1	Model: 10 repeats gave 1 payment. SQLite, 20 simultaneous deliveries: 1 row, 19 absorbed.	MODEL-RUN DB-DEMO
T3	Approved refund fails at gateway	Refund Failed; order not Closed; retry succeeds with prior approval.	FR10, INV2	Model: as predicted. Unapproved execution refused.	MODEL-RUN gateway adapter test planned
T4	Dispatch without payment or pick	Rejected; state unchanged.	FR06	Model: rejected; order record identical before and after.	MODEL-RUN
T5	Courier request fails, then retried	Order stays Paid; intent Pending; retry reuses the existing shipment.	FR11, ADR02	Model: reply lost after creation; retry found existing shipment; 1 shipment.	MODEL-RUN real adapter planned
T6	Customer/support approves refund; cross-customer read	All rejected at the API boundary; support view masked.	FR01, NFR04	Model: customer, support and picker approvals rejected; other customer's read rejected.	MODEL-RUN real endpoint test planned

ID	Scenario	Prediction (written first)	Enforced by	Observed	Evidence
T7	Late payment after reservation expiry	Order flagged for refund if unit resold; stock not oversold.	FR08	Model: released once; unit resold; payment gives Needs Refund and a Requested refund.	MODEL-RUN
T8	Tampered client price	Server price used; non-positive quantity rejected.	FR04	Model: total 118.00 for 2 units with tampered price 0.01.	MODEL-RUN

5.4 Validation evidence



Figure 5.1 Unit test run of the in-memory model: T1 to T8 and S1 to S4 (real run).

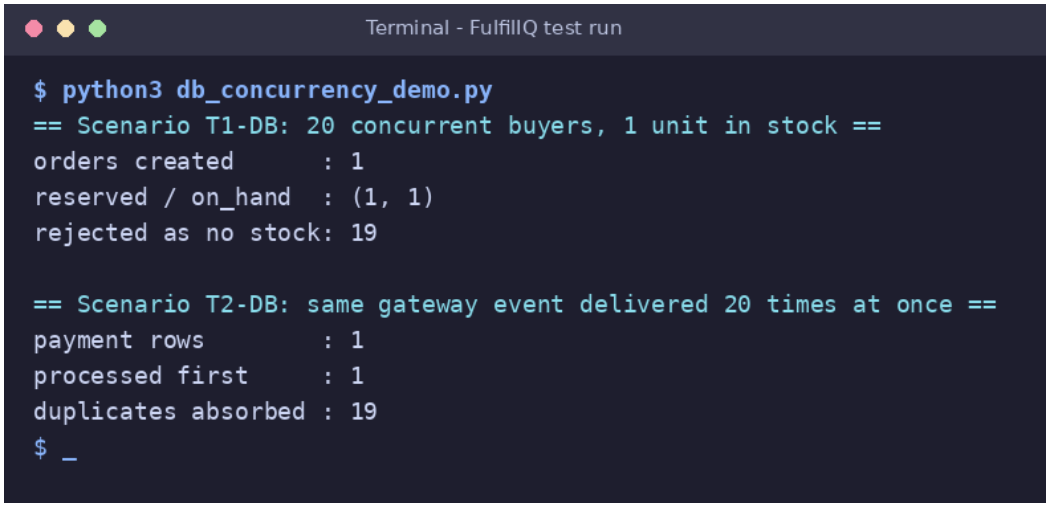


Figure 5.2 Concurrent threads against a real SQL engine (SQLite): last-unit race and duplicate callbacks (real run).

DB-DEMO *SQLite demonstrates the mechanism (conditional UPDATE and UNIQUE key). It is not PostgreSQL, so a concurrent test against the production database is still required.*

5.5 Weakness found and design revision

Item	Content
Weakness 1	Expiry walkthrough (T7): a callback arriving after the reservation was released could mark the order Paid although the unit had been resold, overselling stock.
Revision	On a late callback the system re-reserves if stock remains; otherwise it records the payment, sets the order to Needs Refund and starts the approval-gated refund path. FR08, the order state model (Figure 3.4) and scenario T7 were updated.
Weakness 2	While building the model, a Failed refund could not be retried. Corrected so failure keeps the prior approval (Figure 3.5, scenario T3).
Evidence	T7 and T3 pass after the revision (Figure 5.1). Before the revision, T7's assertion of Needs Refund would have failed because no such state existed.

5.6 Code: validation scenarios

test_fulfiliq.py - Task 5 scenarios T1 to T8

```
def make_shop(on_hand=1):
    s = FulfilIQ()
    for uid, role in [("alice", "customer"), ("bob", "customer"), ("sam", "support"),
                      ("fin", "finance"), ("pat", "picker"), ("mgr", "manager"),
                      ("adm", "admin")]:
        s.add_user(uid, role)
    s.add_product("SKU-1", 50.00, on_hand)
    return s



def pay(s, order, event="E1", gw="GW"):
    return s.confirm_payment(gw, event, order["id"], order["total"],
                             sign(gw, event, order["id"], order["total"]))



class Task5(unittest.TestCase):
    def test_T1_last_unit_two_buyers(self):
        s = make_shop(on_hand=1)
        v = s.stock["SKU-1"]["version"]
        s.place_order("alice", "SKU-1", 1, expected_version=v)
        with self.assertRaisesRegex(Rejected, "stale"):
            s.place order("bob", "SKU-1", 1, expected version=v)
        with self.assertRaisesRegex(Rejected, "out of stock"):
            s.place_order("bob", "SKU-1", 1)
        self.assertEqual(len(s.orders), 1)
        self.assertGreaterEqual(s.available("SKU-1"), 0)

    def test_T2_duplicate_callback_ten_times(self):
        s = make_shop(on_hand=5)
        o = s.place_order("alice", "SKU-1", 1)
        results = [pay(s, o) for _ in range(10)]
        self.assertEqual(len(s.payments), 1)
        self.assertEqual(len(s.intents), 1)
        self.assertEqual(sum(1 for a in s.audit if a["action"] == "payment.confirmed"), 1)
        self.assertEqual([r["duplicate"] for r in results].count(True), 9)

    def test_T3_refund_fails_at_gateway_then_retry(self):
        s = make_shop(on_hand=5)
        o = s.place_order("alice", "SKU-1", 1); pay(s, o)
        s.pick_confirm("pat", o["id"]); s.dispatch("pat", o["id"]); s.mark_delivered(o["id"])
        rid = s.request_refund("sam", o["id"], 59.0)
        with self.assertRaises(Rejected):           # unapproved cannot execute
            s.execute_refund("fin", rid)
        s.approve_refund("fin", rid)
        r = s.execute_refund("fin", rid, gateway_ok=False)
        self.assertEqual(r["state"], "Failed")
        self.assertNotEqual(s.orders[o["id"]]["state"], "Closed")
        self.assertEqual(r["approved_by"], "fin")    # approval kept
        r = s.execute_refund("fin", rid, gateway_ok=True)
        self.assertEqual((r["state"], s.orders[o["id"]]["state"]), ("Executed", "Closed"))

    def test_T4_dispatch_without_payment(self):
        s = make_shop(on_hand=5)
        o = s.place_order("alice", "SKU-1", 1)
        before = dict(o)
```

```
        with self.assertRaises(Rejected):
            s.dispatch("pat", o["id"])
        self.assertEqual(s.orders[o["id"]], before)

    def test_T5_courier_timeout_then_retry(self):
        s = make_shop(on_hand=5)
        o = s.place_order("alice", "SKU-1", 1); pay(s, o)
        c = FakeCourier(); c.fail_after_create = 1
        it = s.sync_courier(o["id"], c)
        self.assertEqual(it["status"], "Pending")
        self.assertEqual(s.orders[o["id"]]["state"], "Paid")   # local order safe
        it = s.sync_courier(o["id"], c)
        self.assertEqual(it["status"], "Done")
        self.assertEqual(len(c.shipments), 1)                  # one logical shipment

    def test_T6_unauthorised_actions(self):
        s = make_shop(on_hand=5)
        o = s.place_order("alice", "SKU-1", 1); pay(s, o)
        s.pick_confirm("pat", o["id"]); s.dispatch("pat", o["id"]); s.mark_delivered(o["id"])
        rid = s.request_refund("sam", o["id"], 59.0)
        for actor in ("alice", "sam", "pat"):
            with self.assertRaises(Rejected):
                s.approve_refund(actor, rid)
        with self.assertRaises(Rejected):
            s.read_order("bob", o["id"])                       # cross-customer read
        view = s.read order("sam", o["id"])
        self.assertNotIn("total", view)
        self.assertTrue(view["contact"].startswith("a***@"))   # masked

    def test_T7_late_payment_after_expiry(self):
        t = [1000.0]
        s = FulfilIQ(clock=lambda: t[0])
        for uid, role in [("alice", "customer"), ("bob", "customer"), ("fin", "finance")]:
            s.add_user(uid, role)
        s.add_product("SKU-1", 50.00, 1)
        o = s.place_order("alice", "SKU-1", 1)
        t[0] += 16 * 60
        s.expire_reservations()
        s.expire_reservations()                                # released exactly once
        self.assertEqual(s.stock["SKU-1"]["reserved"], 0)
        s.place_order("bob", "SKU-1", 1)                       # unit resold
        r = pay(s, o)
        self.assertEqual(r["outcome"], "needs_refund")
        self.assertEqual(s.orders[o["id"]]["state"], "Needs Refund")
        self.assertEqual(s.refunds[next(iter(s.refunds))]["state"], "Requested")
        self.assertGreaterEqual(s.available("SKU-1"), 0)

    def test T8 price and tax ignore client price(self):
        s = make_shop(on_hand=5)
        o = s.place_order("alice", "SKU-1", 2, client_price=0.01)   # tampered price
        self.assertEqual(o["total"], 118.00)                        # 50*2*1.18
        with self.assertRaises(Rejected):
            s.place_order("alice", "SKU-1", 0)                      # non-positive qty
```

db_concurrency_demo.py (SQLite, real threads)

```
"""Real concurrent threads against a real SQL engine (SQLite, not PostgreSQL).
Shows the two DB mechanisms chosen in ADR01: conditional UPDATE + UNIQUE key."""
import os, sqlite3, tempfile, threading

path = os.path.join(tempfile.mkdtemp(), "demo.db")
setup = sqlite3.connect(path)
setup.executescript("""
CREATE TABLE stock(sku TEXT PRIMARY KEY, on_hand INT, reserved INT DEFAULT 0,
                   CHECK (reserved >= 0 AND reserved <= on_hand));
CREATE TABLE orders(id INTEGER PRIMARY KEY, buyer TEXT, sku TEXT);
CREATE TABLE payment_event(gateway_id TEXT, event_id TEXT, order_id INT,
                           PRIMARY KEY (gateway_id, event_id));
INSERT INTO stock(sku, on_hand) VALUES ('SKU-1', 1);
""")
setup.commit(); setup.close()



def connect():
    c = sqlite3.connect(path, timeout=30, isolation_level=None)
    return c



def buy(buyer, results):
    c = connect()
    c.execute("BEGIN IMMEDIATE")
    cur = c.execute("UPDATE stock SET reserved = reserved + 1 "
                    "WHERE sku='SKU-1' AND on_hand - reserved >= 1")      # conditional
    if cur.rowcount == 1:
```

```
        c.execute("INSERT INTO orders(buyer, sku) VALUES (?, 'SKU-1')", (buyer,))
        results.append((buyer, "ORDER CREATED"))
    else:
        results.append((buyer, "OUT OF STOCK"))
    c.execute("COMMIT"); c.close()



def callback(n, results):
    c = connect()
    try:
        c.execute("INSERT INTO payment_event VALUES ('GW','EVT-42', 1)")  # UNIQUE key
        results.append((n, "first delivery -> processed"))
    except sqlite3.IntegrityError:
        results.append((n, "duplicate -> returned existing result"))
    c.close()



def run(target, args_list):
    res = []
    ts = [threading.Thread(target=target, args=(a, res)) for a in args_list]
    [t.start() for t in ts]; [t.join() for t in ts]
    return res



print("== Scenario T1-DB: 20 concurrent buyers, 1 unit in stock ==")
r = run(buy, [f"buyer{i:02d}" for i in range(20)])
ok = [x for x in r if x[1] == "ORDER CREATED"]
c = connect()
print("orders created      :", c.execute("SELECT COUNT(*) FROM orders").fetchone()[0])
print("reserved / on_hand  :", c.execute("SELECT reserved,on_hand FROM stock").fetchone())
print("rejected as no stock:", len(r) - len(ok))

print("\n== Scenario T2-DB: same gateway event delivered 20 times at once ==")
r = run(callback, range(20))
print("payment rows        :", c.execute("SELECT COUNT(*) FROM payment_event").fetchone()[0])
print("processed first     :", sum(1 for x in r if x[1].startswith("first")))
print("duplicates absorbed :", sum(1 for x in r if x[1].startswith("duplicate")))
```

Task 6 Redesign for Expansion and Partial Failure

A. Impact of the changed requirement

Planning figure	Before	After	Factor
Regions	3	12	4x
Simultaneous users	50	500	10x
Stock, payment and scan events	100 / min (1.7 per s)	1,000 / min (16.7 per s)	10x
Assumed peak (5x average)	8.3 per s	83.3 per s	10x

These are design assumptions, not measured capacity.

Affected item	Impact	Change
FR04, FR08, FR10	Tax, shipping and refund limits differ by region.	Regional policy records, versioned (new FR13).
FR11, FR03	Scanners may be offline; stock events arrive late and repeated.	Bounded buffer, stable event IDs, server dedupe (new FR14).
FR10	Approvals captured offline must not look final.	Pending until server validation (new FR15).
FR12	Consolidated reports across 12 regions compete with checkout.	Summary tables refreshed on a schedule.
NFR01, NFR02, NFR03	Higher load stresses checkout latency, deduplication and recovery.	Re-test at new workload; indexes, pagination, queue.
NFR07 (new)	Expansion workload needs an explicit target.	500 users, 1,000 events/min, 12 regions; verify by load test.
Components	Stock, report and integration components are the bottlenecks; the database is the shared limit.	Profile first, then change (see D).

B. Revised architecture

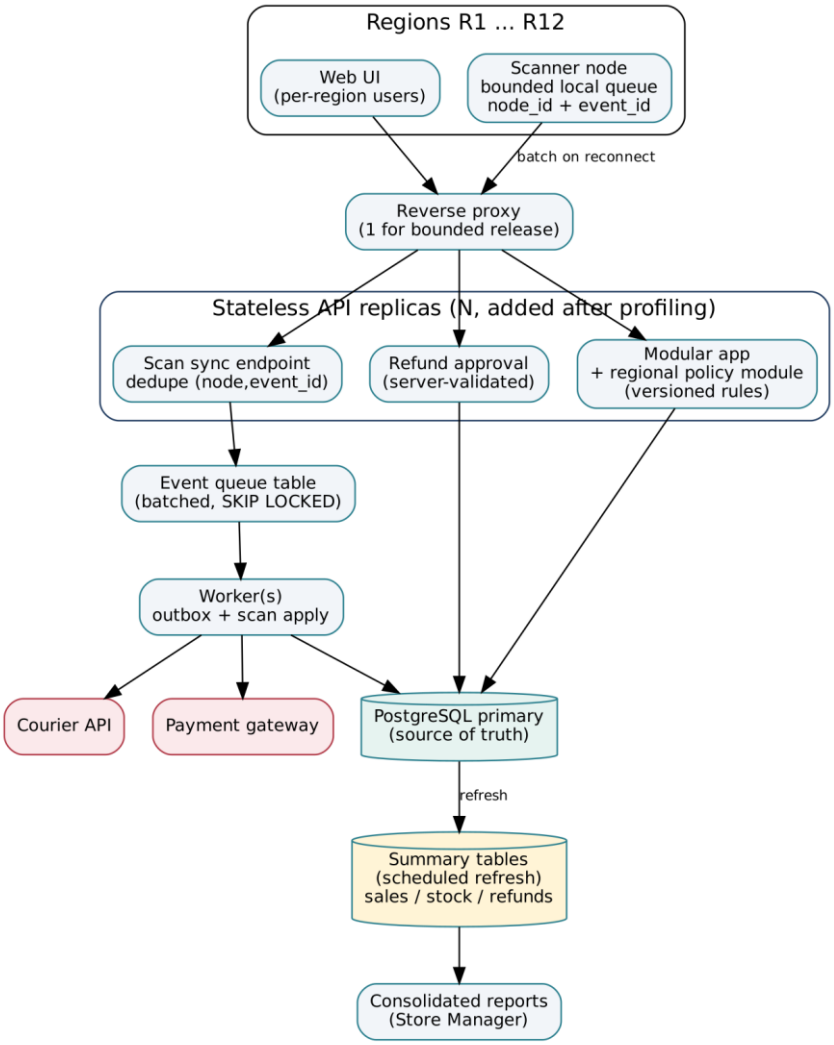


Figure 6.1 Revised architecture for 12 regions, buffered scanners and consolidated reporting.

Component	Responsibility after the change	Linked requirement
Regional policy module	Versioned tax, shipping and refund-limit records per region; each order and event stores the policy version it used.	FR13
Scan sync endpoint	Accepts batches, deduplicates by (node_id, event_id), applies stock changes, flags stale or conflicting events.	FR14, INV3
Event queue table + workers	Absorbs bursts; workers claim rows in batches (FOR UPDATE SKIP LOCKED).	NFR07
Refund approval	Only authenticated server validation can mark a queued approval Approved.	FR15, INV2
Summary tables	Sales, stock and refund summaries refreshed on a schedule for Store Manager reports.	FR12, NFR01
Stateless API replicas	Added only after profiling shows CPU-bound API; database and integration stay explicit.	NFR01

Synchronization rules

1. **SR1 Identity.** Every scan event carries node_id, a stable event_id (node and sequence), observation time, policy version and schema version.

2. **SR2 Bounded buffer.** A scanner queue has a maximum length. When full, scanning stops with a visible error rather than dropping events silently.
3. **SR3 Retain until acknowledged.** Events stay in the local queue until the server acknowledges the batch; replays are therefore expected.
4. **SR4 Deduplicate first.** The server checks (node_id, event_id) before any stock change; a repeated event has no effect.
5. **SR5 Stale observations.** Events older than the stale threshold are marked stale and evaluated by server rules; they are not shown as live counts.
6. **SR6 Invariant guard.** An event that would push on-hand below reserved units is not applied; it is flagged for human stock correction.
7. **SR7 Policy version.** The policy version recorded at capture is kept; a later policy change does not rewrite history.
8. **SR8 Server-validated approval.** A locally captured refund approval is stored as Pending server validation and displayed as Pending until the server authenticates the actor and validates role, amount and state.

C. Validation of the redesign

ID	Scenario	Required observation	Observed	Evidence
S1	Replayed scan burst after reconnect (500 events, replayed in full)	One stock change per node and event ID; bounded queue.	First pass applied 500; replay gave 500 duplicates; on-hand 500; 501st scan raised BufferFull.	MODEL-RUN
S2	Regional policy changes during buffered operation	Event keeps its policy version; current policy advances.	Event stayed at v1 while policy moved to v2; event applied.	MODEL-RUN
S3	Approval queued while disconnected	Shown Pending until server validation; invalid actor rejected.	Support approval stayed Pending; finance approval displayed Approved only after validation.	MODEL-RUN
S4	Stale and conflicting scan	Flagged for human correction; stock unchanged.	1 stale, 1 flagged, on-hand unchanged at 2.	MODEL-RUN
L1	Burst replay at design workload (1,000 events/min for 5 min, 20% replays, 12 nodes)	One change per event; no loss.	5,000 unique applied, 1,000 duplicates absorbed; model time 4.0 ms.	MODEL-RUN timing not capacity
P1	One API replica unavailable	Other replica serves; database intact.	Not run: needs deployed environment.	PLANNED

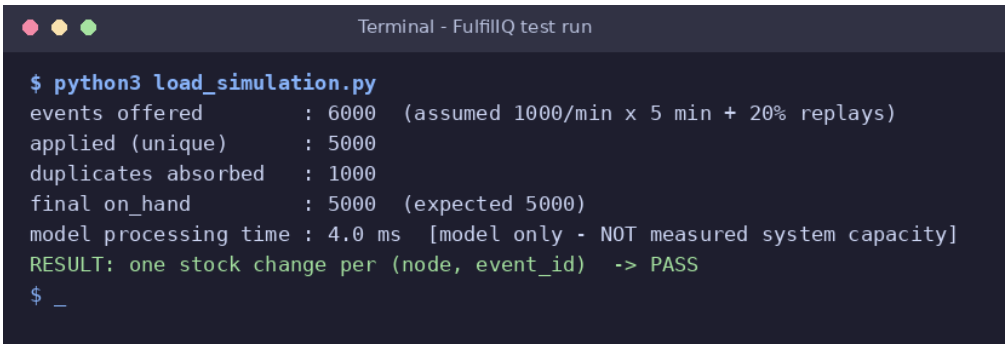


Figure 6.2 Replay of the design workload through the model (real run). The timing describes this model in this environment only.

MODEL-RUN *Design targets are not measured capacity. The load check shows deduplication logic at the planned volume, not throughput of the real system.*

D. Defence of the design

Change considered	Adopted?	Reason (linked to a limitation)
Separate service per module	No	Adds deployment and monitoring cost; no measured bottleneck needs independent scaling yet.
Second API replica behind the proxy	After profiling	Helps only if the API is the bottleneck; does not remove a database or external-service limit.
Indexes and pagination	Yes	Cheap fix for queue and list queries; choose indexes from measured query plans.
Event queue table with batching	Yes	Absorbs 83 events/s peak assumption without a broker.
Summary tables for reports	Yes	Keeps 12-region reporting from competing with checkout.
Read replica or regional databases	Deferred	Costly; consider after measured read load or latency evidence.
Message broker	No	Outbox and queue table meet the need with one fewer system to run.

Aspect	Defence
Benefits	Local availability for scanning and order capture; no lost or double-counted events; regional rules without releases; reports do not slow checkout.
Costs	A scheduled summary job, queue workers to monitor, and policy versioning logic; a possible second API replica.
Consistency	Strong consistency for orders, payments and refunds (database transactions). Eventual consistency for scanner stock and reports, with stale data labelled.
Availability	Scanners and order capture continue during outages. A central database failure still stops checkout and approvals: offline scanning does not imply offline global approval.
Remaining limitations	One database remains a central dependency; capacity numbers are assumptions; long offline periods increase conflicting scans needing human correction; no regional data-residency analysis.

E. Code: scanner buffer, deduplication and server-validated approval

fulfiliq_test_model.py - ScannerNode and displayed_status

```
class BufferFull(Exception):
    pass


class ScannerNode:
    """Disconnected warehouse scanner: bounded, uniquely identified buffer."""

    def __init__(self, node_id, max_queue=1000, clock=time.time):
        self.node_id, self.clock = node_id, clock
        self.queue, self.max_queue, self.seq = deque(), max_queue, 0
        self.approval_queue = []

    def scan(self, sku, delta, policy_version=1, observed_at=None):
        if len(self.queue) >= self.max_queue:             # never drop silently
            raise BufferFull("local queue full: stop scanning and sync")
        self.seq += 1
        ev = {"node": self.node_id, "event_id": f"{self.node_id}-{self.seq:06d}",
              "sku": sku, "delta": delta, "policy_version": policy_version,
              "observed_at": observed_at or self.clock(), "schema": 1}
        self.queue.append(ev)
        return ev

    def queue_approval(self, actor, refund_id):
        entry = {"actor": actor, "refund_id": refund_id, "status": "Pending server validation"}
        self.approval_queue.append(entry)
        return entry

    def flush(self):
        batch = list(self.queue)
        return batch                                      # kept until server acks

    def ack(self, n):
```

```
        for _ in range(n):
            self.queue.popleft()


def displayed_status(entry):
    """The UI may only show 'Approved' after server validation."""
    return "Approved" if entry["status"] == "Approved" else "Pending"
```

fulfiliq_test_model.py - apply_scan_events and validate_queued_approval

```
    def apply_scan_events(self, events, now=None):
        now = now or self.clock()
        applied = dup = stale = 0
        for e in events:
            key = (e["node"], e["event_id"])
            if key in self.scan_seen:
                dup += 1
                continue
            self.scan_seen.add(key)
            if now - e["observed_at"] > STALE_SECONDS:
                stale += 1
                e = {**e, "stale": True}
            s = self.stock[e["sku"]]
            if s["on_hand"] + e["delta"] < s["reserved"]:  # would break invariant 3
                self.discrepancies.append(e)               # human stock correction
                continue
            s["on_hand"] += e["delta"]
            s["version"] += 1
            applied += 1
        return {"applied": applied, "duplicates": dup, "stale": stale,
                "flagged": len(self.discrepancies)}

    def validate_queued_approval(self, entry):
        """Server-side check of an approval captured while disconnected."""
        try:
            self.approve_refund(entry["actor"], entry["refund_id"])
            entry["status"] = "Approved"
        except Rejected as exc:
            entry["status"] = "Rejected by server"
            entry["reason"] = str(exc)
        return entry
```

test_fulfiliq.py - scenarios S1 to S4

```
class Task6(unittest.TestCase):
    def test_S1_replayed_scan_burst_after_reconnect(self):
        s = make_shop(on_hand=0)
        node = ScannerNode("N1", max_queue=500)
        for _ in range(500):
            node.scan("SKU-1", +1)
        batch = node.flush()
        first = s.apply_scan_events(batch)
        replay = s.apply_scan_events(batch)                    # reconnect replays all
        self.assertEqual(first["applied"], 500)
        self.assertEqual(replay["duplicates"], 500)
        self.assertEqual(s.stock["SKU-1"]["on_hand"], 500)     # one change per event id
        with self.assertRaises(BufferFull):                    # bounded buffer
            node.scan("SKU-1", +1)

    def test_S2_policy_change_during_buffered_operation(self):
        s = make_shop(on_hand=10)
        v1 = s.set_policy("adm", "R1", 0.18, 100.0)
        node = ScannerNode("N2")
        ev = node.scan("SKU-1", +1, policy_version=v1)
        v2 = s.set_policy("adm", "R1", 0.12, 150.0)            # changes while offline
        self.assertNotEqual(ev["policy_version"], v2)          # event keeps its version
        self.assertEqual(s.policy("R1")["version"], v2)
        self.assertEqual(s.apply_scan_events([ev])["applied"], 1)

    def test_S3_queued_approval_not_shown_as_approved(self):
        s = make_shop(on_hand=5)
        o = s.place_order("alice", "SKU-1", 1); pay(s, o)
        s.pick_confirm("pat", o["id"]); s.dispatch("pat", o["id"]); s.mark_delivered(o["id"])
        rid = s.request_refund("sam", o["id"], 59.0)
        node = ScannerNode("N3")
        e_ok = node.queue_approval("fin", rid)
        e_bad = node.queue_approval("sam", rid)
        self.assertEqual(displayed_status(e_ok), "Pending")    # before validation
        s.validate_queued_approval(e_bad)
        self.assertEqual(displayed_status(e_bad), "Pending")
        s.validate_queued_approval(e_ok)
        self.assertEqual(displayed_status(e_ok), "Approved")

    def test_S4_stale_and_conflicting_scans_are_flagged(self):
        s = make_shop(on_hand=2)
```

```
        s.place_order("alice", "SKU-1", 2)                      # reserved = 2
        old = ScannerNode("N4").scan("SKU-1", -2, observed_at=time.time() - 3600)
        res = s.apply_scan_events([old])
        self.assertEqual(res["stale"], 1)
        self.assertEqual(res["flagged"], 1)                     # needs human correction
        self.assertEqual(s.stock["SKU-1"]["on_hand"], 2)        # invariant 3 holds
```

load simulation.py

```
"""Replays a design-assumption workload through the in-memory model.
Timings describe THIS model on THIS container, not system capacity."""
import random, time
from fulfiliq test model import FulfilIQ, ScannerNode

EVENTS_PER_MIN, MINUTES, DUP_RATE = 1000, 5, 0.20
s = FulfilIQ(); s.add_product("SKU-1", 10, 0)
nodes = [ScannerNode(f"N{i}", max_queue=5000) for i in range(12)]   # 12 regions
for i in range(EVENTS_PER_MIN * MINUTES):
    nodes[i % 12].scan("SKU-1", +1)
batch = [e for n in nodes for e in n.flush()]
batch += random.Random(7).sample(batch, int(len(batch) * DUP_RATE))  # replays
random.Random(1).shuffle(batch)

t0 = time.perf_counter()
res = {"applied": 0, "duplicates": 0}
for i in range(0, len(batch), 100):                      # batches of 100
    r = s.apply_scan_events(batch[i:i + 100])
    res["applied"] += r["applied"]; res["duplicates"] += r["duplicates"]
dt = time.perf_counter() - t0
print(f"events offered        : {len(batch)}  (assumed {EVENTS_PER_MIN}/min x {MINUTES} min +
    {int(DUP_RATE*100)}% replays)")
print(f"applied (unique)      : {res['applied']}")
print(f"duplicates absorbed   : {res['duplicates']}")
print(f"final on_hand         : {s.stock['SKU-1']['on_hand']}  (expected
    {EVENTS_PER_MIN*MINUTES})")
print(f"model processing time : {dt*1000:.1f} ms  [model only - NOT measured system capacity]")
assert s.stock["SKU-1"]["on_hand"] == EVENTS_PER_MIN * MINUTES
print("RESULT: one stock change per (node, event_id)  -> PASS")
```

Task 7 Individual Technical Reflection

Requirement with the greatest influence

FR05 (one gateway and event identity creates at most one payment) shaped more of the design than any other requirement. It produced the UNIQUE key in the schema (Task 4), the idempotent response in the sequence diagram (Figure 3.3), the outbox design in ADR02 and the stable event identities in the scanner rules (SR1 and SR4). The SQLite run (Figure 5.2) showed that 20 simultaneous deliveries of one event leave exactly one payment row.

Most difficult stakeholder conflict

Fast refunds versus finance control (C4) was the hardest. Support wants quick customer relief; finance wants accountability; and the limit that divides them is still unresolved (U1). The resolution lets agents request but never approve, and lets a manager approve below the limit. I am least sure the limit and the manager role match the real organisation, which is why they remain open questions.

Inconsistency found through modeling and validation

Connecting the sequence diagram to the state model showed that a browser success page had been treated as payment, and the T3 scenario showed that a Failed refund could not be retried. The expiry walkthrough (T7) found the most serious one: a late callback could mark a Cancelled order Paid after the unit was resold. That required a Needs Refund state and a new path in Figures 3.4 and 3.5.

Unresolved or significant trade-off

Centralised consistency is simple and keeps invariants in single transactions, but it makes one database a central dependency for checkout and approvals. Durable integration improves local availability, yet introduces delayed synchronization and worker operations. I have not resolved when the load justifies a read replica or a service split; that needs measurement.

Evidence needed before implementation

- Real order and stock query patterns, to choose indexes and size summary tables.
- A concurrent last-unit and duplicate-callback test on PostgreSQL, not SQLite.
- Gateway callback behaviour (retry timing, signature format) and courier timeout behaviour from their sandboxes.
- Outage history, to set realistic NFR03 recovery and NFR08 availability targets.
- A load test with stated dataset, warm-up and duration to check NFR01 and NFR07, and tests of the real endpoints for T6.
- Answers to U1 to U4 from the business.

Quick trace for oral verification

Requirement	UML element	Component	Scenario	Invariant / note
FR05	PaymentEvent; Figure 3.3	Payment callback handler	T2 (model, SQLite)	INV1: one gateway/event identity, one payment.
FR11	IntegrationIntent; Figure 3.3 steps 6-14	Outbox worker	T5	Failed courier sync stays Pending and is recoverable by correlation key.

Acknowledgement of sources and tools

Scenario and assignment text: CSA10 Software Engineering, Assignment 2 (SIMATS Engineering). Tools used: Python 3 (unittest, sqlite3), Graphviz, matplotlib and Pillow for diagrams and terminal captures, and docx for this report. Jira and Confluence are the selected tracking and documentation

tools; Figures 2.3 and 2.4 are mock-ups pending the author's own screenshots. Drafting and code were produced with the assistance of an AI model (Claude, Anthropic), and the author is responsible for reviewing and verifying every claim before submission. Stakeholder responses are simulated.

Appendix A Complete In-Memory Test Model

File: fulfiliq_test_model.py (synthetic data only). Run the checks with: python3 -m unittest -v

fulfiliq_test_model.py

```
"""FulfilIQ in-memory test model (synthetic data only).

Demonstrates RULE LOGIC for the order-fulfilment design. It is NOT the
production system: no network, no real gateway/courier, no card data.
"""
import hashlib
import hmac
import threading
import time
from collections import deque

SECRET = b"synthetic-test-secret"
REFUND_LIMIT = 100.00          # planning assumption (unresolved question U1)
RESERVATION_SECONDS = 15 * 60  # planning assumption (unresolved question U2)
STALE_SECONDS = 15 * 60        # scan observations older than this are "stale"

ROLE_PERMS = {
    "customer": {"order.place", "order.read_own"},
    "picker": {"pick.confirm", "order.dispatch", "picklist.read"},
    "support": {"order.read_masked", "refund.request", "return.start"},
    "finance": {"order.read_masked", "refund.approve", "refund.execute"},
    "manager": {"report.read", "refund.approve.small"},
    "admin": {"policy.write", "user.manage"},
}


class Rejected(Exception):
    """A command was refused; state must be unchanged."""


class CourierTimeout(Exception):
    pass


def sign(gateway_id, event_id, order_id, amount):
    msg = f"{gateway id}|{event id}|{order id}|{amount:.2f}".encode()
    return hmac.new(SECRET, msg, hashlib.sha256).hexdigest()


class FakeCourier:
    """Controlled test interface (status: SIMULATED, not a live courier)."""

    def   init  (self):
        self.shipments = {}          # correlation_key -> tracking id
        self.fail_after_create = 0   # create shipment, then lose the reply

    def lookup(self, key):
        return self.shipments.get(key)

    def create(self, key):
        tracking = self.shipments.setdefault(key, f"TRK-{len(self.shipments) + 1:04d}")
        if self.fail after create > 0:
            self.fail_after_create -= 1
            raise CourierTimeout("reply lost after creation")
        return tracking


class FulfilIQ:
    def __init__(self, clock=time.time):
        self.clock = clock
        self.lock = threading.RLock()   # stands in for a DB transaction
        self.catalog, self.stock = {}, {}
        self.users, self.orders, self.reservations = {}, {}, {}
        self.payments, self.refunds, self.intents = {}, {}, {}
        self.audit, self.scan_seen, self.discrepancies = [], set(), []
        self.policies = {}              # region -> list of versioned policies
        self._seq = 0

    # ---------- helpers ----------
    def _id(self, prefix):
        self._seq += 1
        return f"{prefix}{self._seq:04d}"
```

```
    def _log(self, actor, action, order_id, version=None):
        self.audit.append({"actor": actor, "action": action,
                           "order": order_id, "at": self.clock(), "version": version})

    def _need(self, user, perm):
        role = self.users.get(user, {}).get("role")      # identity from session
        if perm not in ROLE_PERMS.get(role, set()):
            raise Rejected(f"role '{role}' lacks {perm}")

    # ---------- setup ----------
    def add_user(self, uid, role, region="R1"):
        self.users[uid] = {"role": role, "region": region}

    def add_product(self, sku, price, on_hand):
        self.catalog[sku] = price
        self.stock[sku] = {"on_hand": on_hand, "reserved": 0, "version": 1}

    def set_policy(self, actor, region, tax_rate, refund_limit):
        self._need(actor, "policy.write")
        versions = self.policies.setdefault(region, [])
        v = len(versions) + 1
        versions.append({"version": v, "tax": tax_rate, "refund_limit": refund_limit})
        return v

    def policy(self, region):
        versions = self.policies.get(region)
        return versions[-1] if versions else {"version": 0, "tax": 0.18,
                                              "refund_limit": REFUND_LIMIT}

    def available(self, sku):
        s = self.stock[sku]
        return s["on_hand"] - s["reserved"]

    # ---------- FR02 / FR03 / FR04 ----------
    def place_order(self, user, sku, qty, expected_version=None, client_price=None):
        self._need(user, "order.place")
        if qty <= 0:
            raise Rejected("quantity must be positive")
        with self.lock:                                  # one transaction
            s = self.stock[sku]
            if expected_version is not None and expected_version != s["version"]:
                raise Rejected("stale stock version")
            if self.available(sku) < qty:                # conditional update
                raise Rejected("out of stock")
            s["reserved"] += qty
            s["version"] += 1
            region = self.users[user]["region"]
            pol = self.policy(region)
            total = round(self.catalog[sku] * qty * (1 + pol["tax"]), 2)  # FR04
            oid = self._id("O")
            self.orders[oid] = {"id": oid, "owner": user, "sku": sku, "qty": qty,
                                "state": "Placed", "total": total, "version": 1,
                                "region": region, "policy_version": pol["version"],
                                "email": f"{user}@example.test"}
            self.reservations[oid] = {"qty": qty, "sku": sku, "active": True,
                                      "expires_at": self.clock() + RESERVATION_SECONDS}
            self._log(user, "order.placed", oid, 1)
            return self.orders[oid]

    # ---------- FR08 ----------
    def _release(self, oid):
        r = self.reservations[oid]
        if r["active"]:                                  # released exactly once
            self.stock[r["sku"]]["reserved"] -= r["qty"]
            self.stock[r["sku"]]["version"] += 1
            r["active"] = False

    def expire_reservations(self):
        with self.lock:
            for oid, o in self.orders.items():
                r = self.reservations[oid]
                if o["state"] == "Placed" and r["active"] and r["expires_at"] <= self.clock():
                    self._release(oid)
                    o["state"] = "Cancelled"
                    o["version"] += 1
                    self._log("scheduler", "order.expired", oid, o["version"])

    # ---------- FR05 ----------
    def confirm_payment(self, gateway_id, event_id, order_id, amount, signature):
        if not hmac.compare_digest(signature, sign(gateway_id, event_id, order_id, amount)):
            raise Rejected("signature invalid")
        key = (gateway_id, event_id)
        with self.lock:
            if key in self.payments:                     # idempotent replay
```

```
            return {**self.payments[key], "duplicate": True}
        o = self.orders.get(order_id)
        if o is None:
            raise Rejected("unknown order")
        if abs(amount - o["total"]) > 0.005:
            raise Rejected("amount mismatch")
        result = {"order": order_id, "outcome": "paid", "duplicate": False}
        if o["state"] == "Placed":
            o["state"] = "Paid"
        elif o["state"] == "Cancelled":               # late callback (FR08)
            sku, qty = o["sku"], o["qty"]
            if self.available(sku) >= qty:            # re-reserve if possible
                self.stock[sku]["reserved"] += qty
                self.reservations[order_id]["active"] = True
                o["state"] = "Paid"
                result["outcome"] = "paid_after_rereserve"
            else:                                     # unit already resold
                o["state"] = "Needs Refund"
                rid = self._id("R")
                self.refunds[rid] = {"id": rid, "order": order_id, "amount": o["total"],
                                     "state": "Requested", "approved_by": None,
                                     "requested_by": "system", "version": 1}
                result["outcome"] = "needs_refund"
        else:
            raise Rejected(f"payment not valid in state {o['state']}")
        o["version"] += 1
        self.payments[key] = result
        if o["state"] == "Paid":                      # outbox intent, same txn
            self.intents[order_id] = {"correlation_key": f"ship-{order_id}",
                                      "status": "Pending", "attempts": 0,
                                      "external_id": None}
        self._log("gateway", "payment.confirmed", order_id, o["version"])
        return result

    # ---------- FR06 / FR07 / FR09 ----------
    def pick_confirm(self, user, order_id):
        self._need(user, "pick.confirm")
        with self.lock:
            o = self.orders[order_id]
            if o["state"] != "Paid":
                raise Rejected(f"cannot pack from {o['state']}")
            o["state"] = "Packed"; o["version"] += 1
            self._log(user, "order.packed", order_id, o["version"])

    def dispatch(self, user, order_id):
        self._need(user, "order.dispatch")
        with self.lock:
            o = self.orders[order_id]
            paid = any(p["order"] == order_id and p["outcome"].startswith("paid")
                       for p in self.payments.values())
            if o["state"] != "Packed" or not paid:
                raise Rejected("dispatch needs confirmed payment and pick confirmation")
            o["state"] = "Dispatched"; o["version"] += 1
            self._log(user, "order.dispatched", order_id, o["version"])

    def mark_delivered(self, order_id):
        with self.lock:
            o = self.orders[order_id]
            if o["state"] != "Dispatched":
                raise Rejected("only dispatched orders can be delivered")
            o["state"] = "Delivered"; o["version"] += 1

    # ---------- FR11 outbox worker ----------
    def sync_courier(self, order_id, courier):
        with self.lock:
            it = self.intents[order_id]
            if it["status"] == "Done":
                return it
            it["attempts"] += 1
        existing = courier.lookup(it["correlation_key"])  # reconcile first
        try:
            tracking = existing or courier.create(it["correlation_key"])
        except CourierTimeout:
            return it                                 # stays Pending
        with self.lock:
            it["status"], it["external_id"] = "Done", tracking
        return it

    # ---------- FR10 refunds ----------
    def request_refund(self, user, order_id, amount):
        self._need(user, "refund.request")
        o = self.orders[order_id]
        if o["state"] != "Delivered":
            raise Rejected("refund needs an eligible delivered order")
        rid = self._id("R")
        self.refunds[rid] = {"id": rid, "order": order_id, "amount": amount,
```

```
                              "state": "Requested", "approved_by": None,
                              "requested_by": user, "version": 1}
        self._log(user, "refund.requested", order_id)
        return rid

    def approve_refund(self, user, refund_id):
        role = self.users.get(user, {}).get("role")
        r = self.refunds[refund_id]
        limit = self.policy(self.orders[r["order"]]["region"])["refund_limit"]
        allowed = (role == "finance") or (role == "manager" and r["amount"] <= limit)
        if not allowed:
            raise Rejected(f"role '{role}' may not approve this refund")
        if r["state"] != "Requested":
            raise Rejected(f"refund is {r['state']}")
        r["state"], r["approved_by"] = "Approved", user
        r["version"] += 1
        self._log(user, "refund.approved", r["order"])

    def execute_refund(self, user, refund_id, gateway_ok=True):
        self._need(user, "refund.execute")
        r = self.refunds[refund_id]
        if r["approved_by"] is None or r["state"] not in ("Approved", "Failed"):
            raise Rejected("refund has no approval")      # invariant 2
        if not gateway_ok:
            r["state"] = "Failed"                         # approval is KEPT
            self._log(user, "refund.failed", r["order"])
            return r
        r["state"] = "Executed"
        o = self.orders[r["order"]]
        o["state"] = "Closed"; o["version"] += 1
        self._log(user, "refund.executed", r["order"])
        return r

    # ---------- FR01 / privacy ----------
    def read_order(self, user, order_id):
        o = self.orders[order_id]
        role = self.users.get(user, {}).get("role")
        if role == "customer":
            if o["owner"] != user:
                raise Rejected("not your order")
            return {k: o[k] for k in ("id", "state", "total")}
        if role in ("support", "finance"):
            name, domain = o["email"].split("@")
            return {"id": o["id"], "state": o["state"],
                    "contact": name[0] + "***@" + domain}   # masked, no payment data
        raise Rejected("role may not read orders")

    # ---------- Task 6: scans, buffered approvals ----------
    def apply_scan_events(self, events, now=None):
        now = now or self.clock()
        applied = dup = stale = 0
        for e in events:
            key = (e["node"], e["event_id"])
            if key in self.scan_seen:
                dup += 1
                continue
            self.scan_seen.add(key)
            if now - e["observed_at"] > STALE_SECONDS:
                stale += 1
                e = {**e, "stale": True}
            s = self.stock[e["sku"]]
            if s["on_hand"] + e["delta"] < s["reserved"]:  # would break invariant 3
                self.discrepancies.append(e)               # human stock correction
                continue
            s["on_hand"] += e["delta"]
            s["version"] += 1
            applied += 1
        return {"applied": applied, "duplicates": dup, "stale": stale,
                "flagged": len(self.discrepancies)}

    def validate_queued_approval(self, entry):
        """Server-side check of an approval captured while disconnected."""
        try:
            self.approve_refund(entry["actor"], entry["refund_id"])
            entry["status"] = "Approved"
        except Rejected as exc:
            entry["status"] = "Rejected by server"
            entry["reason"] = str(exc)
        return entry


class BufferFull(Exception):
    pass
```

```
class ScannerNode:
    """Disconnected warehouse scanner: bounded, uniquely identified buffer."""

    def __init__(self, node_id, max_queue=1000, clock=time.time):
        self.node_id, self.clock = node_id, clock
        self.queue, self.max_queue, self.seq = deque(), max_queue, 0
        self.approval_queue = []

    def scan(self, sku, delta, policy_version=1, observed_at=None):
        if len(self.queue) >= self.max_queue:             # never drop silently
            raise BufferFull("local queue full: stop scanning and sync")
        self.seq += 1
        ev = {"node": self.node_id, "event_id": f"{self.node_id}-{self.seq:06d}",
              "sku": sku, "delta": delta, "policy_version": policy_version,
              "observed at": observed at or self.clock(), "schema": 1}
        self.queue.append(ev)
        return ev

    def queue_approval(self, actor, refund_id):
        entry = {"actor": actor, "refund_id": refund_id, "status": "Pending server validation"}
        self.approval_queue.append(entry)
        return entry

    def flush(self):
        batch = list(self.queue)
        return batch                                      # kept until server acks

    def ack(self, n):
        for _ in range(n):
            self.queue.popleft()



def displayed status(entry):
    """The UI may only show 'Approved' after server validation."""
    return "Approved" if entry["status"] == "Approved" else "Pending"
```